

# A Cross-Layer Security Survey of LLM Agentic Systems: Attacks, Defenses, and Open Challenges

KAZI SHAHARAIR SHARIF, Kennesaw State University, USA

SHIVA SHRESTHA, Kennesaw State University, USA

ELISA BERTINO, Purdue University, USA

JIADONG LOU, Rowan University, USA

DAVID WU, Kennesaw State University, USA

HONGHUI XU, Kennesaw State University, USA

Large language models (LLMs) are evolving from passive text generators into autonomous agents that execute real-world actions via tool use, persistent memory, and multi-agent coordination. This shift expands the security landscape: attacks can now trigger unauthorized actions such as data exfiltration, code execution, and financial transactions. Existing work primarily studies isolated vulnerabilities and lacks a unified framework for analyzing cross-layer attack propagation and defense deployment. This survey presents a cross-stack analysis of security threats and defenses for LLM-based agent systems. We introduce a five-layer deployment stack covering infrastructure, model inference, tools, orchestration, and user interaction, and define four trust boundaries that capture security assumptions. We formalize a unified threat model with six attacker types and five security properties, and categorize 13 attack classes, including prompt injection, tool abuse, memory poisoning, supply-chain attacks, and multi-agent manipulation. We organize existing defenses into four generations, from prompt-level heuristics to architectural mechanisms, and evaluate them using eight deployment criteria, including latency overhead, utility impact, and robustness to adaptive adversaries. Our analysis reveals a clear gap between attack severity and defense maturity: critical agent-specific threats, such as memory poisoning, tool supply-chain exploitation, and inter-agent manipulation, lack effective and deployable defenses. By unifying attacks, defenses, and deployment constraints within a single architectural framework, this survey provides a foundation for securing LLM-based agents in practice. We conclude by identifying six open problems and outlining directions for robust and trustworthy agentic systems.

CCS Concepts: • **Security and privacy** → **Distributed systems security**; **Software security engineering**; *Intrusion/anomaly detection and malware mitigation*; • **Computing methodologies** → **Natural language processing**; **Multi-agent systems**; • **Information systems** → *Information retrieval*.

Additional Key Words and Phrases: Large Language Models, LLM Agents, AI Agents, Agent Security, Prompt Injection, Tool Abuse, Memory Poisoning, supply-chain attacks, Multi-Agent Systems, Adversarial Machine Learning, Privacy, Trustworthy AI, Deployment Security, Attack Taxonomy, Defense Taxonomy

## 1 Introduction

Large language models (LLMs) have rapidly evolved from text generation systems into autonomous agents that act in real-world environments: beyond producing language, they retrieve documents, invoke APIs, maintain persistent memory, send emails, execute code, coordinate with other agents, and make decisions with real operational consequences. Systems such as OpenAI Assistants[66], Anthropic Claude, Microsoft Copilot[61], and open source frameworks like LangChain [15] and AutoGen [97] are already being integrated into enterprise workflows, customer service, software development, and financial operations. This shift from passive language processing to active task execution has fundamentally transformed the security landscape. Whereas a compromised chatbot may generate harmful text, a compromised agent has stronger capabilities to carry out

---

Authors' Contact Information: Kazi Shaharair Sharif, Kennesaw State University, Kennesaw, USA, ksharif2@students.kennesaw.edu; Shiva Shrestha, Kennesaw State University, Kennesaw, USA, sshres16@students.kennesaw.edu; Elisa Bertino, Purdue University, West Lafayette, USA, bertino@purdue.edu; Jiadong Lou, Rowan University, Glassboro, USA, louj@rowan.edu; David Wu, Kennesaw State University, Kennesaw, USA, dwu10@students.kennesaw.edu; Honghui Xu, Kennesaw State University, Kennesaw, USA, hxu10@kennesaw.edu.

harmful actions and cause more serious consequences. Moreover, each component of an agent system, including tool interfaces, memory stores, planning logic, and multi-agent communication, introduces new points of entry that do not arise in standalone LLMs, significantly broadening the attack surface. The OWASP Top 10 for LLM Applications identifies prompt injection as the most critical vulnerability, appearing in more than 73% of production deployments assessed through security audits [28, 68]. Agent Security Bench further reports that mixed attacks achieve an 84.3% success rate across agent-based scenarios [106], while Lupinacci et al. [60] show that 82.4% of models can be compromised through inter-agent communication, an attack vector that has no counterpart in conventional chatbot settings.

To understand this new threat landscape of LLM agents, three closely connected questions need to be answered. The first concerns the adversarial setting: who can attack the system, what capabilities they possess, and which assets must be protected. Answering this question establishes the threat model and defines the security objectives of the analysis. The second concerns the attack surface and attack mechanisms: what concrete attacks are possible, how they exploit different components of the agentic workflow, and how effective they are under different assumptions. This leads to an attack taxonomy that organizes existing threats by their entry points, mechanisms, and impacts. The third concerns mitigation: what defenses are available, how well they counter these attacks, and whether they can be realistically deployed in production systems. This forms the defense taxonomy, which evaluates countermeasures not only by their effectiveness but also by their deployability under real-world constraints. Together, these three questions provide a unified analytical framework that connects adversarial assumptions, concrete attack behaviors, and practical defense strategies for securing LLM agents.

Crucially, these elements cannot be analyzed in isolation; rather, they must be understood across the full architecture of an LLM-agent system. In our proposed five-layer LLM-agent deployment stack, Layer 3 refers to the tool and memory ecosystem, Layer 2 to model inference, Layer 4 to orchestration and planning, and Layer 5 to the user interface and output channel. An attacker who poisons a retrieved document at Layer 3 can exploit the model's inability to separate instructions from data at Layer 2, steer the agent away from its intended plan at Layer 4, and ultimately cause the exfiltration of sensitive information through the output channel at Layer 5. Defending against such cross-layer attack chains, therefore, requires a holistic understanding of the entire deployment stack, from infrastructure and model inference to the tool ecosystem, orchestration layer, and user interface. This deployment-stack perspective serves as the central organizing principle of our survey.

Several recent surveys address aspects of LLM agent security, each contributing valuable insights but leaving significant gaps that our work addresses: He et al. [35] provide case-study-driven analysis of agent-specific threats and privacy concerns but do not organize findings by deployment layer or evaluate defense deployment readiness. Deng et al. [24] identify four knowledge gaps in AI agent security but focus on theoretical gap analysis rather than quantitative defense evaluation. Das et al. [21] comprehensively survey LLM-level security and privacy challenges including membership inference and data poisoning; however, their scope does not extend to agent-specific capabilities such as tool invocation, persistent memory, or multi-agent coordination, which are the primary sources of the threats cataloged in our Sections 5 and 6. Tang et al. [86] present a broad catalog of attacks, defenses, and dual-use applications of LLM agents with unified evaluation criteria; however, they organize by attack type rather than by deployment layer, do not assess deployment-readiness constraints such as latency overhead and computational cost, and do not identify the defense evolution trajectory from prompt-level to architectural approaches. Ferrag et al. [29] advance the frontier by providing the first systematic analysis of protocol-level vulnerabilities in MCP and A2A agent workflows, introducing formal threat models for input manipulation, model compromise, and

protocol exploitation; however, their coverage does not span the full deployment stack, and defense readiness assessment is not part of their framework. Yu et al. [103] examine the trustworthiness of LLM agents from a threats-and-countermeasures perspective, covering a wide range of security and safety concerns; however, they do not provide a layered architectural framework for mapping attacks to system components or evaluate defenses against deployment-readiness criteria. Shahriar et al. [81] present a broad survey of the agentic security landscape, organizing the literature around three main pillars: applications, threats, and defenses, and covering more than 160 papers. However, their scope spans both the use of agents as cybersecurity tools, including offensive and defensive applications, and the security of agent systems themselves. As a result, the survey does not emphasize the generational evolution of defense strategies, nor does it provide a systematic evaluation framework grounded in deployment constraints.

Across these efforts, existing surveys remain fragmented across general LLM security, agent trustworthiness, cybersecurity applications, and protocol-specific analyses. As a result, the field still lacks a unified deployment-oriented framework connecting where vulnerabilities arise in the agent stack, how adversaries exploit them, and whether proposed defenses can be realistically deployed.

Our survey differs from these seven prior studies in several key ways, none addressed in a unified manner by existing work. First, we introduce a five-layer deployment-stack model that maps both attacks and defenses to specific components and trust boundaries, enabling the cross-layer analysis in Section 5.6 of how attacks propagate and compose across layers. Second, we evaluate every defense against eight deployment-readiness criteria—including latency, computational cost, utility impact, false-positive rate, and adaptive resistance—so practitioners can judge fit for a deployment environment rather than attack-success reduction alone (Section 6.1). Third, we trace a four-generation evolution of defenses, from ad hoc prompt engineering toward architectural guarantees, a trajectory that parallels traditional security shifts such as input sanitization to parameterized queries in SQL-injection defense (Section 6). Fourth, we quantify three major gaps: memory poisoning exceeds 90% attack success with no published defense, tool supply-chain attacks reach 77% absent any certification, and multi-agent manipulation compromises 82.4% of models despite only immature proposals. Together, these show that the most severe agent-specific threats have the weakest countermeasures (Sections 5.7 and 8), and our framework unifies where agent vulnerabilities arise, how they propagate, and which defenses are practical under real-world constraints.

We formalize these objectives through three research questions:

- RQ1:** What are the practical attack vectors across the full deployment stack of LLM-based agent systems, and how do they differ from threats to standalone LLMs?
- RQ2:** What defense mechanisms exist for each attack class, and to what extent are they deployable under real-world constraints of latency, cost, scalability, and usability?
- RQ3:** What are the critical gaps in current defenses that prevent secure deployment of LLM agents at scale?

This survey makes the following contributions:

- (1) A five-layer deployment-stack model (L1–L5) with four trust boundaries (TB-1 through TB-4) that organizes agent architecture by function and identifies where security assumptions change (Section 3).
- (2) A unified attacker taxonomy (T1–T6) mapping six adversary types to deployment stack layers, extending prior models that cover only subsets of the threat landscape (Section 4).
- (3) A systematic catalog of 13 attack classes organized by target layer, with quantitative attack success rates from published benchmarks, identifying the most critical agent-specific threats (Section 5).

- (4) A four-generation defense evolution (G1–G4) evaluated against eight deployment-readiness criteria, providing the first systematic assessment of defense viability under production constraints (Section 6).
- (5) Identification of three critical defense gaps and formalization of six open problems for the research community (Section 8).

**Organization.** The remainder of this survey is organized as follows. Section 2 provides background on LLMs, agent architectures, and security preliminaries. Section 3 introduces the five-layer deployment stack, and Section 4 formalizes the threat model. Sections 5 and 6 present the attack and defense taxonomies, respectively. Section 7 provides cross-cutting analysis, Section 8 discusses open challenges and future directions, and Section 9 concludes the survey.

## 2 Background and Preliminaries

### 2.1 Large Language Models (LLMs)

Large Language Models (LLMs) are Transformer-based models that generate text through autoregressive next-token prediction [73, 88]. For agent security, the most important architectural property is the *flat token stream*: system prompts, user queries, retrieved documents, tool outputs, and other contextual inputs are concatenated into a single token sequence with no built-in distinction between instructions and data [115]. As formalized in Section 4, this limitation underlies prompt injection and confused-deputy failures, because adversarial instructions embedded in external content may be processed as legitimate commands. Modern LLMs are also shaped by safety alignment and tool-use mechanisms. Alignment methods such as RLHF, Constitutional AI, and DPO aim to reduce harmful behavior [8, 9, 67, 74], but they remain vulnerable to competing objectives and mismatched generalization [93]. These limitations are exploited by jailbreaking attacks discussed in Section 5. Function calling further expands the security problem by enabling models to produce structured tool invocations, such as JSON objects specifying an API name and parameters [79]. This shifts the model from a text generator to an actuator: a compromised output may trigger external actions rather than merely produce harmful text.

### 2.2 LLM-Based Agents: Definition and Components

An LLM-based agent combines a core LLM with planning, tool use, memory, and environment interaction. We define an LLM-based agent as a software system in which a large language model orchestrates multi-step task execution by reasoning over observations, invoking external tools, maintaining state, and producing actions that affect its environment [90, 99]. This distinguishes agents from standalone LLMs: where an LLM primarily generates text, an agent can send emails, query databases, execute code, retrieve information, or coordinate with other agents. Deployed examples include OpenAI Assistants [66], Anthropic’s Claude with computer use [40], Microsoft Copilot [61], and open-source frameworks such as LangChain [15, 90] and AutoGen [97].

**Planning and Reasoning.** Agents decompose high-level goals into executable steps using reasoning and planning strategies such as Chain-of-Thought, ReAct, and Tree-of-Thought [94, 101, 102]. From a security perspective, each intermediate reasoning step is a potential point where adversarial input can alter the agent’s plan, as examined in Section 5.

**Tool Interfaces.** Agents act on their environment through tool invocation, where the LLM produces structured calls that a runtime executes [79]. Emerging protocols such as MCP and A2A standardize connections to external tools, data sources, and other agents [6, 39, 58, 85]. These interfaces expand agent capability but also create supply-chain and trust assumptions around third-party tools and plugins, which we analyze in Section 5.

**Memory Systems.** Agents use short-term context and long-term memory to maintain state across interactions. Long-term memory is commonly implemented through vector databases or related external stores [69], while Retrieval-Augmented Generation (RAG) injects retrieved passages into the model context [53]. This improves task performance but also allows externally stored or previously injected content to influence future reasoning, making memory integrity a central security concern.

**Multi-Agent Systems.** Complex workflows may involve multiple specialized agents that communicate and coordinate through frameworks such as AutoGen, MetaGPT, and CrewAI [38, 62, 97]. These systems can use peer-to-peer, broadcast, or hierarchical coordination patterns. While multi-agent designs improve task decomposition, they also introduce inter-agent communication as an adversarial channel through which malicious instructions or manipulated state can propagate. Section 3 maps these components to the deployment stack, and Section 5 analyzes the attacks that exploit them.

### 2.3 Security Preliminaries.

Analyzing agent security requires extending established security concepts to systems that combine language understanding, tool interaction, and autonomous decision-making. The foundational framework is the **CIA triad**—Confidentiality, Integrity, and Availability [78]. Because LLM agents can perform real-world actions, we extend this vocabulary with two additional properties used throughout the survey: Safety/Alignment, which concerns whether the agent’s actions remain consistent with user goals, and Privacy, which concerns inappropriate inference or leakage of personal information. These five properties are formalized in Section 4. Several traditional security concepts also take on agent-specific significance. The **confused deputy problem** [34] arises when an authorized program is tricked into misusing its authority; in LLM agents, this occurs when adversarial instructions embedded in retrieved data or tool outputs are processed as legitimate commands. **Privilege escalation** occurs when an attacker causes the agent to perform actions beyond its intended authorization, while **supply-chain vulnerabilities** arise from reliance on third-party tools, plugins, and data sources [65]. Together, these concepts provide the vocabulary for the threat model in Section 4, the attack taxonomy in Section 5, and defense analysis in Section 6.

## 3 Deployment-Stack Architecture Model (Key Contribution)

### 3.1 The Five-Layer Deployment Stack Model

To systematically analyze the security of LLM-based agents, we introduce a five-layer deployment stack that organizes agent components by function and identifies where security assumptions change. This model serves as the structural foundation for the remainder of the survey: Section 5 maps attacks to the layers they target, and Section 6 maps defenses to the layers they protect. Compared with prior surveys that organize threats primarily by attack type [24, 35] or agent component [86], our layered model supports cross-stack analysis, such as how an attack entering through the tool ecosystem at Layer 3 can influence model inference at Layer 2, alter planning at Layer 4, and produce harmful consequences at Layer 5. Figure 1 illustrates the full stack, including attack entry points, trust boundaries, and defense overlays.

**Layer 1: Infrastructure.** Layer 1 contains the hardware, cloud platforms, networking, storage, and runtime environments that support agent execution. Its key assets include model weights, API credentials, inference traffic, and computational resources. The main security concern at this layer is protecting confidentiality and integrity against infrastructure compromise, network interception, side channels, and resource exhaustion while preserving the latency requirements of LLM inference.

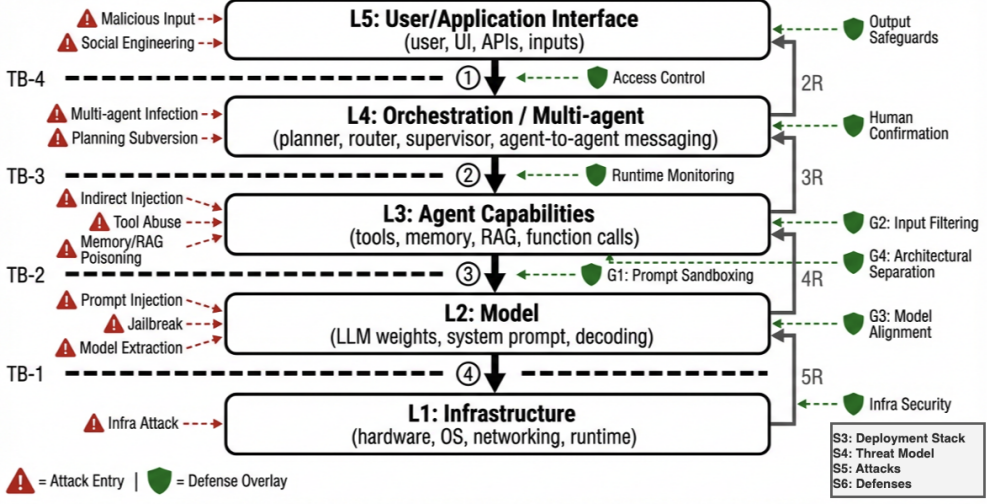


Fig. 1. Five-layer deployment stack for LLM agentic systems with attack entry points, trust boundaries, and defense overlays.

**Layer 2: Model Inference.** Layer 2 contains the core LLM, including the system prompt, decoding process, and safety-alignment mechanisms. Its key assets are the model parameters, safety boundaries, and system instructions that define the agent’s role and permissions. The main security concern is the flat token stream: trusted instructions, user queries, retrieved documents, tool outputs, and memory contents are processed by the same model context, creating the instruction–data confusion that underlies prompt injection and confused-deputy failures.

**Layer 3: Tool and Memory Ecosystem.** Layer 3 mediates the agent’s interaction with external resources, including tools, APIs, plugins, MCP and A2A workflows [39, 58], vector databases[69], and RAG pipelines [53]. Its key assets are tool credentials, memory contents, retrieved documents, and the integrity of tool responses. The main security concern is that Layer 3 is the primary entry point for untrusted external data, which may later be injected into the model context or used to drive tool invocation.

**Layer 4: Orchestration and Planning.** Layer 4 implements the agent’s control loop: decomposing goals, selecting tools, interpreting results, updating plans, and coordinating with other agents. This includes planning methods such as ReAct [102] and Chain-of-Thought [94] and multi-agent frameworks such as AutoGen and MetaGPT [38, 97]. Its key assets are the agent’s plan, task decomposition logic, and inter-agent messages. The main security concern is that compromising this layer compromises the agent’s decision-making process and can redirect otherwise legitimate tools toward adversarial goals.

**Layer 5: User Interface and Output.** Layer 5 handles communication between the agent and users or downstream systems, including user prompts, rendered responses, generated files, emails, and other externally visible actions. Its key assets are output integrity, user privacy, and the correctness of actions performed on the user’s behalf. The main security concern is that attacks originating in lower layers often become visible here as data leakage, misleading outputs, unauthorized messages, or other real-world consequences.

Table 1. Deployment-stack components, assets, and trust boundaries.

| Layer                        | Components                                     | Key Assets                               | Trust Boundary |
|------------------------------|------------------------------------------------|------------------------------------------|----------------|
| <b>L1: Infrastructure</b>    | Cloud, GPU, network, storage                   | Model weights, API keys, compute         | TB-1           |
| <b>L2: Model Inference</b>   | LLM, safety alignment, system prompt           | Model parameters, safety boundaries      | TB-2 ★         |
| <b>L3: Tool &amp; Memory</b> | Tools, MCP/A2A, RAG, vector DB, APIs           | Tool credentials, memory, retrieved docs | TB-3           |
| <b>L4: Orchestration</b>     | Planning (ReAct/CoT), multi-agent coordination | Agent plan, inter-agent messages         | TB-4           |
| <b>L5: User Interface</b>    | Input/output, rendered responses, sent emails  | Output integrity, user privacy           | (User)         |

**Trust Boundaries.** The interfaces between layers define four trust boundaries where security assumptions change:

**TB-1 ( $L1 \leftrightarrow L2$ ): Infrastructure–Model boundary.** TB-1 separates infrastructure from model inference and captures risks such as traffic interception, model-weight tampering, and side-channel exposure.

**TB-2 ( $L2 \leftrightarrow L3$ ): Instructions–Data boundary.** TB-2 separates trusted instructions from untrusted data entering the model context; this is the critical boundary exploited by indirect prompt injection.

**TB-3 ( $L3 \leftrightarrow L4$ ): Agent–Environment boundary.** TB-3 separates the agent from external tools, APIs, memory stores, and data sources, determining whether retrieved or tool-generated content is treated as trustworthy.

**TB-4 ( $L4 \leftrightarrow L5$ ): Agent–Agent boundary.** TB-4 separates agents in multi-agent systems, where messages from peer agents may carry adversarial instructions or manipulated state. Table 1 summarizes the components, assets, and trust boundaries associated with each layer.

### 3.2 Cross-Layer Data Flow

The structural model in Section 3.1 shows where agent components reside; we now trace how information moves across these layers during execution. Consider a user request: “Summarize the latest financial report and email the summary to the team.” Although simple at the interface level, this task engages all five layers and crosses multiple trust boundaries. The request first enters through the user interface at Layer 5 and is passed to the orchestrator at Layer 4, where the agent decomposes it into subtasks: retrieving the report, summarizing its contents, drafting an email, and sending the result. This planning step invokes the LLM at Layer 2, meaning that the user request, system prompt, and intermediate reasoning context are processed within the same token stream. The orchestrator then issues a structured tool call to Layer 3 to retrieve the financial report. The returned document, which may originate from an external database, shared drive, or third-party service, is injected into the model context alongside trusted instructions, crossing the critical instructions–data boundary TB-2. If long-term memory is enabled, the agent may also retrieve prior summaries, correspondence, or user preferences from a vector database through a RAG pipeline [53], further expanding the context with content that may have entered the system in earlier sessions—what we term a *delayed trust-boundary crossing*, since prior-session content influences current reasoning with no re-verification of its integrity. After processing the accumulated context, the model generates a summary, the orchestrator invokes an email-sending tool at Layer 3, the message is dispatched through infrastructure at Layer 1, and the final confirmation appears at Layer 5. In multi-agent settings, the same workflow may be divided among specialized agents, such as a

retrieval agent, writing agent, and review agent; each inter-agent message crosses TB-4 and creates an additional channel for adversarial influence.

This flow reveals four security-critical properties of agent execution. First, *layer amplification*: a single input can influence planning at Layer 4, reasoning at Layer 2, tool use at Layer 3, and final output at Layer 5. Second, *context accumulation*: each step appends new information to the model context, progressively mixing trusted instructions with untrusted tool outputs, retrieved documents, and memory entries. Third, *repeated trust-boundary crossings*: the plan–execute–observe loop repeatedly traverses TB-2 and TB-3, increasing the number of opportunities for adversarial content to affect execution. Fourth, *model-centric privilege mediation*: because the LLM helps select tools, interpret results, and generate actions, compromising its reasoning can compromise the integrity of the entire agent stack.

### 3.3 Relationship to Existing Frameworks

The deployment-stack model introduced in Section 3.1 is complementary to existing practitioner and research frameworks for LLM-agent security. Two frameworks are particularly relevant: MAESTRO, which provides a layered risk-modeling view of agentic AI systems, and the OWASP Top 10 for LLM Applications, which provides a widely adopted vulnerability taxonomy for LLM-based applications.

MAESTRO, developed by the Cloud Security Alliance [43], offers a layered threat-modeling framework for secure, trustworthy, and reliable agentic AI systems. It identifies risks across the agent lifecycle, including model capabilities, tool integration, orchestration, and multi-agent collaboration. Our five-layer deployment stack shares MAESTRO’s core insight that agent security must be analyzed across system layers rather than as a monolithic problem. However, our model has a different analytical goal: instead of primarily cataloging risks, it defines explicit trust boundaries (TB-1 through TB-4) and uses them to explain how attacks propagate across layers. This enables the cross-layer attack-chain analysis in Section 5 and supports the defense-readiness evaluation in Section 6. The OWASP Top 10 for LLM Applications [68] provides a practitioner-oriented taxonomy of major vulnerabilities in LLM applications. Our attack taxonomy encompasses the OWASP categories but reorganizes them according to where they arise in the deployment stack. This layer-based organization makes it possible to analyze how a vulnerability at one layer, such as indirect prompt injection through retrieved data, can propagate through model inference, planning, tool invocation, and output generation. Our taxonomy also emphasizes agent-specific threats that are less fully captured by general LLM vulnerability lists, including memory poisoning, tool supply-chain attacks, and multi-agent manipulation. Overall, MAESTRO contributes layered risk modeling, and OWASP contributes operational vulnerability guidance. Our framework connects these perspectives by integrating three elements: a deployment-oriented architecture, a quantitative attack taxonomy with cross-layer propagation analysis, and a defense taxonomy evaluated under deployment-readiness constraints. This allows practitioners not only to identify where agent risks arise, but also to assess which defenses are viable for specific deployment contexts.

## 4 Threat Model

Section 3 introduced the five-layer deployment stack and its trust boundaries. We now formalize the threat model by defining three elements: the adversaries that exploit these boundaries, the security properties that must be preserved, and the architectural root cause of the most critical agent attacks. Together, these elements frame the attack taxonomy in Section 5 and the defense analysis in Section 6. Figure 2 maps the six attacker types and five security properties onto the deployment stack, with TB-2 highlighted as the instruction–data boundary analyzed in Section 4.3.



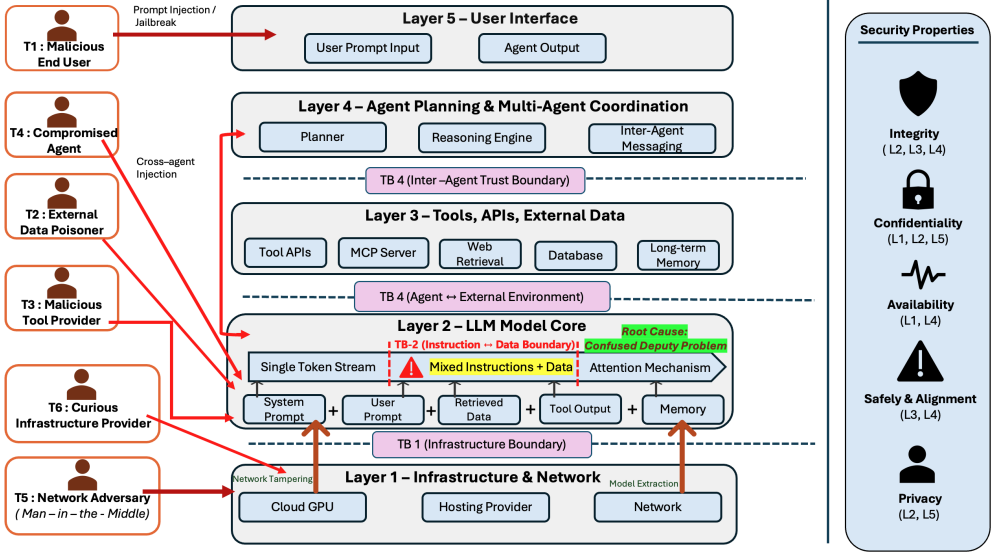


Fig. 2. Unified threat model for LLM agentic systems, mapping attacker types, security properties, trust boundaries, and the instruction-data boundary onto the five-layer deployment stack.

#### 4.1 Attacker Taxonomy

We classify adversaries into six types based on their access level, capabilities, and the deployment stack layers they can reach. This taxonomy extends prior attacker models — which typically consider only a subset of these types into a unified framework that spans the full deployment stack. The Agent Security Bench (ASB) considers attacks across system prompts, user prompts, memory, and tool usage but does not model supply-chain or infrastructure adversaries [106]. AgentDojo formalizes a black-box data poisoner which controls content the agent retrieves but does not address multi-agent or network-level threats[23]. The MAC framework defines an attacker with white-box knowledge of the system architecture and toolset, without the ability to directly interfere with runtime execution [47]. Our taxonomy unifies and extends these models as follows.

**T1: Malicious End User.** T1 interacts with the agent through the standard user interface and can perform direct prompt injection, jailbreaking, or system-prompt extraction. Although this attacker has no access to internal tools, memory, or infrastructure, a successful compromise can trigger unauthorized tool calls and real-world actions rather than merely harmful text.

**T2: External Data Poisoner.** T2 does not directly interact with the agent but controls content that the agent may later retrieve, such as web pages, emails, shared documents, database entries, or other Layer 3 data sources. This makes T2 especially consequential for agent systems: the attacker can plant adversarial payloads passively and wait until the agent retrieves them during normal execution. Because prompt injection is ranked as the top LLM application vulnerability by OWASP [68], this attacker class captures a central risk at the intersection of external data, retrieval, and the instruction-data boundary.

**T3: Malicious Tool or Plugin Provider.** T3 supplies a compromised tool, MCP server, API endpoint, or plugin that the agent invokes. Such an attacker can return malicious tool outputs, alter tool descriptions, or exfiltrate information passed as tool parameters. The rapid growth of agent

Table 2. Attacker taxonomy with capabilities and target layers.

| Type | Name                    | Access        | Capabilities                         | Layers  | Boundary   |
|------|-------------------------|---------------|--------------------------------------|---------|------------|
| T1   | Malicious End User      | Prompt (L5)   | Injection, jailbreak, extraction     | L2, L5  | —          |
| T2   | Data Poisoner           | Controls data | Indirect injection (docs, web, APIs) | L3 → L2 | TB-2, TB-3 |
| T3   | Tool Provider           | Tool/plugin   | Malicious outputs, manipulation      | L3      | TB-3       |
| T4   | Compromised Agent       | Multi-agent   | Cross-agent attack, escalation       | L4      | TB-4       |
| T5   | Network Adversary       | MITM (L1)     | Interception, tampering              | L1      | TB-1       |
| T6   | Infrastructure Provider | Cloud (L1)    | Extraction, side-channel             | L1, L2  | TB-1       |

tool ecosystems without mature certification or vetting processes makes this threat increasingly realistic.

**T4: Compromised Agent.** T4 is an agent within a multi-agent system that has been compromised through prior injection, adversarial design, or poisoned fine-tuning data. It can send malicious messages to peer agents, manipulate shared state, and exploit inter-agent trust. Lupinacci et al. [60] show that this threat is highly effective: 82.4% of tested models were compromised through inter-agent communication, compared with 41.2% through direct prompt injection, indicating that current safety training addresses human-to-AI trust more effectively than AI-to-AI trust.

**T5: Network Adversary.** T5 occupies a man-in-the-middle position and can observe, intercept, or tamper with traffic between the agent, cloud services, tools, or other agents. This enables eavesdropping on inference traffic, modifying API responses, or inferring sensitive information from communication patterns.

**T6: Curious Infrastructure Provider.** T6 is an honest-but-curious cloud or hosting provider with physical or logical access to the infrastructure supporting the agent. Although it does not actively tamper with execution, it may observe GPU memory, activations, inference patterns, or other side-channel signals, creating confidentiality and privacy risks. Table 2 summarizes the attacker taxonomy with capabilities and target layers.

## 4.2 Security Properties

The CIA triad introduced in Section 2.3 provides a starting vocabulary, but the autonomous, action-oriented nature of LLM agents requires an extended set of security properties. Table 3 summarizes the five properties used in this survey and maps them to the deployment-stack layers where violations primarily occur. Integrity, confidentiality, and availability follow the classical CIA triad, while safety/alignment and privacy capture agent-specific risks introduced by tool use, autonomous action, and contextual inference. Availability also includes agent-specific disruption patterns such as recursive or contagious blocking in multi-agent systems [110], while privacy includes inference-time leakage through outputs, tool calls, or reasoning traces [30, 82]. These properties serve as evaluation criteria for the defenses in Section 6 and are used throughout Section 5 to characterize the impact of each attack class.

## 4.3 The Root Cause: The Confused Deputy Problem

The attacker taxonomy defines who attacks, and the security properties define what must be protected; this subsection explains why these attacks succeed. The root cause is an architectural limitation of current LLMs: system prompts, user queries, tool outputs, retrieved documents, and memory contents are processed as a single undifferentiated token sequence. Because the model has no built-in mechanism for separating instructions from data, adversarial instructions

Table 3. Mapping between security properties and deployment stack layers.

| Property         | Definition                              | Violated When                                                   | Primary Layers |
|------------------|-----------------------------------------|-----------------------------------------------------------------|----------------|
| Integrity        | Actions match user intent               | Injection causes unauthorized tool-calls or plan deviation      | L2, L3, L4     |
| Confidentiality  | Sensitive data not exposed              | Data exfiltrated via output, side channel, or tool parameters   | L1, L2, L5     |
| Availability     | System remains operational              | DoS, resource exhaustion, contagious blocking                   | L1, L4         |
| Safety/Alignment | No unintended real-world harm           | Agent executes adversarial instructions causing harmful actions | L3, L4         |
| Privacy          | Personal data not inferred from outputs | CoT leaks private data; tool-calls reveal user context          | L2, L5         |

embedded in external content may be interpreted as legitimate commands. Zverev et al. [115] provide empirical evidence that even state-of-the-art LLMs cannot reliably enforce this distinction when both appear in the same context. This limitation creates a confused deputy vulnerability [34]. In agent systems, the deputy is the agent itself: it holds delegated user privileges over tools, memory, and communication channels, but may be tricked into misusing those privileges when malicious instructions are embedded in retrieved documents, tool responses, or stored memories. Greshake et al. [31] demonstrated this failure mode in LLM-integrated applications, where hidden web-page instructions caused agents to exfiltrate user data. Within the deployment stack, the confused deputy problem appears at TB-2, the instruction–data boundary at Layer 2. Each time untrusted Layer 3 content, such as tool outputs, retrieved documents, or memory entries, is concatenated with trusted instructions in the model context, TB-2 is crossed. Because agent workflows repeatedly traverse this boundary during the plan,execute and observe loop, each crossing creates an opportunity for adversarial data to influence behavior. This has direct implications for defense design. Model-level defenses attempt to train the LLM to resist manipulation, and detection-based defenses try to filter adversarial content before it reaches the model. However, both approaches still rely on the model or a detector to manage an unresolved architectural ambiguity. Architectural defenses address the root cause more directly by externalizing the instruction–data boundary into the system design rather than relying on the model to enforce it internally. This shift from symptom management to architectural enforcement is a central theme of this survey.

#### 4.4 Scope and Assumptions

**In scope**, we consider attacks specific to or significantly amplified by agent capabilities—tool use, persistent memory, multi-step planning, and multi-agent coordination—and evaluate defenses with published implementations or empirical evaluations against the deployment-readiness criteria of Section 6, covering both single- and multi-agent architectures from cloud to edge. **Out of scope**, we exclude attacks on standalone LLMs without tool access or agent capabilities, which are well covered by existing LLM-safety surveys [21], as well as robotic or embodied agents where physical safety dominates; purely theoretical defenses without implementation are noted but not assessed in depth. We **assume** a pre-trained, instruction-tuned base LLM accessed via API or deployed locally; an attacker who cannot modify model weights at inference time (a separate class of training-time attacks); a system prompt and configuration set by a trusted developer; and at least one source of external input (a data source, tool, or end-user input), the minimum condition for the agent-specific threats we survey.

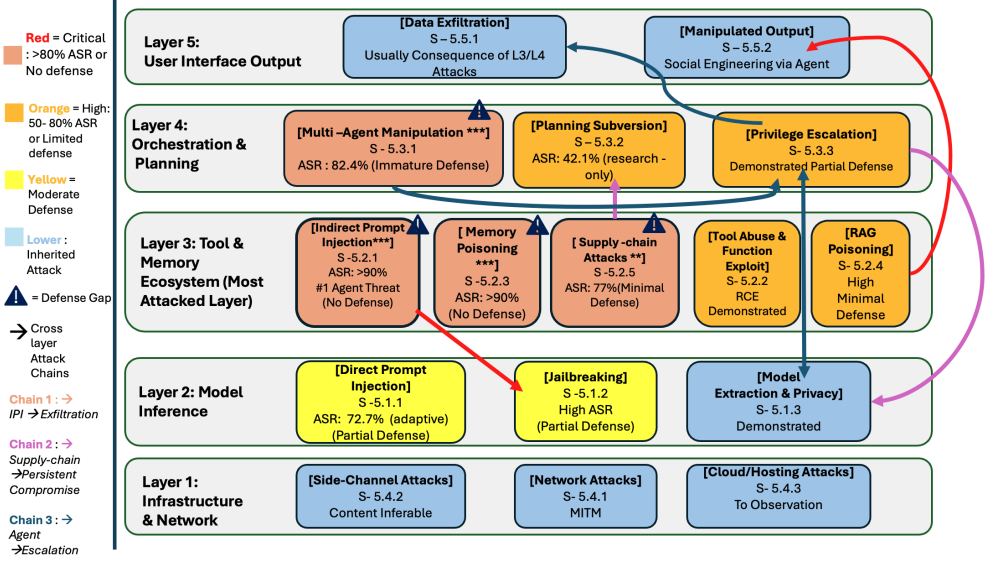


Fig. 3. Cross-layer attack taxonomy for LLM agentic systems mapped onto the five-layer deployment stack.

#### 4.5 Why the Agent Threat Model Differs from Chatbot Security

A natural question is whether existing LLM safety research for chatbots and text generation is sufficient for agent security. We argue that it is not, for five reasons. First, the consequences differ: chatbot attacks usually produce harmful text, whereas agent attacks can trigger harmful actions such as sending emails, executing code, deleting files, or initiating transactions. Second, the attack surface is broader: agents receive input not only from users but also from tools, retrieved documents, memory, APIs, and other agents. Third, attacks can persist: poisoned memory entries may be retrieved and reused across later workflows, extending the impact beyond the original interaction. Fourth, attacks can amplify: a single injection can cascade through the plan-execute-observe loop and trigger multiple tool calls. Fifth, trust is more complex: agent systems involve users, models, tools, memory stores, external data sources, and other agents, each introducing assumptions that an adversary may exploit. Quantitative evidence supports this distinction. Li et al. [54] show that commercial LLM agents are vulnerable to simple attacks whose impact is amplified by agent capabilities. Agent Security Bench reports an 84.3 percent attack success rate for mixed attacks spanning multiple channels [106], while Lupinacci et al. [60] find that 82.4 percent of models are compromised through inter-agent communication, an attack vector absent from chatbot settings. These results motivate our deployment-stack approach: agent security requires analyzing attacks and defenses across the full architectural stack, as Sections 5 and 6 undertake.

### 5 Attack Taxonomy (Organized by Deployment Stack Layer)

Building on the deployment stack in Section 3 and the threat model in Section 4, we classify attacks by their target layer (L1–L5), attacker type (T1–T6), and violated security properties. This layer-based organization, rather than the attack-type organization used in previous surveys [24, 35, 86], directly mirrors the defense taxonomy in Section 6 and enables the cross-layer analysis in Section 5.6. Figure 3 illustrates the complete attack taxonomy mapped onto the deployment stack.

## 5.1 Layer 2 Attacks: Model Inference

Layer 2 attacks target the LLM itself, including its generation process, safety alignment, and learned parameters. Although these attacks also affect standalone LLMs, agent systems amplify their impact because compromised model behavior can trigger tool calls, planning errors, privacy leakage, or external actions.

### 5.1.1 Direct Prompt Injection.

**Attacker: T1 (Malicious End User) | Properties violated: Integrity, Safety/Alignment.** Direct prompt injection occurs when a user crafts input that overrides the system prompt through the standard input channel at Layer 5. Just as SQL injection exploits the mixing of code and data in database queries, prompt injection exploits the mixing of instructions and data in the flat token stream (Section 4.3). Rossi et al. [71, 75] provide an early categorization of these attacks, Schulhoff et al. [80] present the first systematic study, and subsequent work has documented dozens of injection templates [106]. In chatbots, successful injection produces harmful text; in agents, it can trigger tool-calls, file operations, financial transactions, or code execution—an escalation from harmful text to harmful actions that is the fundamental reason chatbot-era defenses are insufficient for agents (Section 4.5). Agent Security Bench reports a 72.7 percent direct-injection ASR across ten agent scenarios [106]. Direct injection remains relevant as the simplest attack to execute and a common first step in multi-stage attacks; defenses are discussed in Sections 6.1 (prompt-level) and 6.1.3 (model-level).

### 5.1.2 Jailbreaking.

**Attacker: T1 (Malicious End User) | Properties violated: Safety/Alignment.** Jailbreaking attacks bypass the safety alignment mechanisms discussed in Section 2.1, causing the model to produce outputs it was trained to refuse. Wei et al. [93] identify two core failure modes: competing objectives, where helpfulness conflicts with safety, and mismatched generalization, where safety training fails to cover deployment-time inputs. Existing techniques include gradient-based adversarial suffixes such as GCG [113], automated jailbreak generation such as AutoDAN [56], multi-turn escalation such as Crescendo [77], many-shot jailbreaking over long contexts [1, 5], and adaptive random-search attacks [4]. In agent systems, jailbreaking is more severe than content-safety failure because it can bypass guardrails governing tool use, enabling unsafe actions such as sending emails, executing code, or transferring funds. Mitigations are discussed in Section 6.1.3, but AutoInject’s 22% ASR against SecAlign shows that adaptive adversaries remain effective [18].

### 5.1.3 Model Extraction and Privacy Attacks.

**Attacker: T6 (Infrastructure Provider), T5 (Network Adversary) | Properties violated: Confidentiality, Privacy.** Model-level privacy attacks extract sensitive information from the LLM or its training data. Carlini et al. [12] showed that LLMs memorize and can regurgitate training data, including PII, through targeted prompting, and subsequent work established scaling laws showing larger models memorize more [11]. System-prompt extraction against real-world commercial applications [44] recovers the hidden instructions governing an agent’s behavior and permissions, exposing its security configuration. Agent architectures add new dimensions: Green et al. [30] show that chain-of-thought reasoning—a cornerstone of agent planning (Section 2.2)—leaks private information through intermediate traces even when the final output is sanitized, a concern that is acute for reasoning-intensive models such as OpenAI’s o1 series. Additionally, split-inference deployments (part on-device, part in the cloud) expose intermediate activations at the network

boundary, creating side-channel opportunities at Layer 1. These vectors motivate infrastructure-level defenses (Section 6.3), including trusted execution environments and encrypted inference, alongside output-level privacy mechanisms.

## 5.2 Layer 3 Attacks: Tool and Memory Ecosystem

Layer 3 is where agents diverge most clearly from standalone LLMs. Tools, memory, APIs, and external data sources create agent-specific attack surfaces that chatbots do not have. For this reason, Layer 3 contains several of the most critical threats in our taxonomy, including indirect injection, memory poisoning, tool abuse, and supply-chain compromise.

### 5.2.1 Indirect Prompt Injection.

**Attacker: T2 (External Data Poisoner), T3 (Tool Provider) | Properties violated: Integrity, Safety/Alignment, Confidentiality.** Indirect prompt injection is one of the most critical agent-specific attack classes. Unlike direct injection (Section 5.1.1), where a malicious user supplies harmful input directly, indirect injection embeds adversarial instructions within external data the agent retrieves during execution: the attacker never interacts with the agent but contaminates its data environment and waits for retrieval. This exploits the confused deputy vulnerability (Section 4.3)—the model treats retrieved content and trusted instructions equivalently, so embedded directives execute with the agent’s full privileges. Greshake et al.[31] first formalized this attack, showing how hidden web-page instructions could cause LLM-integrated applications to exfiltrate user data and manipulate plugins. It has since been systematized across the channels through which external data enters the agent’s context.

**Injection Channels.** Injection channels reflect the diversity of Layer 3 data sources. Web pages are the highest-risk channel: Johnson et al.[50] show that hidden instructions in HTML accessibility attributes (e.g., aria-labels) induce targeted agent behavior at rates exceeding 90 percent across major websites, because web agents parse the accessibility tree rather than the rendered page. Email and messaging are equally vulnerable: hidden instructions in an email are processed automatically by the victim’s assistant, with no link clicked or attachment opened[31]. RAG documents can be poisoned to activate on retrieval[23]; tool and API responses from malicious services (T3) can hijack reasoning mid-task; database description fields can carry instructions surfaced during queries; and multimodal inputs such as images with invisible text overlays are an emerging channel as vision-language agents spread.

**Quantitative Evidence.** Indirect injection has been measured across benchmarks with increasingly sophisticated techniques. AgentDojo [23], the most comprehensive dynamic benchmark, reports a 53.1 percent baseline ASR using simple "Important Message" templates, with task completion dropping from 69 to 45 percent under attack. ChatInject [13] exploits multi-turn chat-template formatting to reach 32 percent ASR on AgentDojo and 52 percent on InjecAgent [105]. Most alarmingly, AutoInject [18] trains a 1.5B-parameter RL policy to generate adaptive injections, achieving 78 percent ASR on Gemini-2.5-Flash and 22 percent against SecAlign-70B, the strongest model-level defense—showing that model-level defenses alone are insufficient against adaptive adversaries. Attacks are also maturing toward protocol-level analysis: Guo et al. [32] systematize MCP security, framing the Model Context Protocol as a new attack surface and organizing diverse vectors into a unified framework that moves beyond single-case demonstrations. Section 6 organizes defenses against indirect injection into four generations—prompt-level (6.1.1), detection-based (6.1.2), model-level (6.1.3), and architectural (6.1.4)—a progression that is a central theme of that section.

### 5.2.2 Tool Abuse and Function Call Exploitation.

**Attacker: T1 (Malicious End User), T2 (Data Poisoner) | Properties violated: Integrity, Safety/Alignment.** Beyond injection, agents face misuse of their tool-calling capabilities. Tool abuse occurs when an agent is manipulated into calling tools with unintended parameters, calling tools it should not access, or chaining calls to achieve outcomes no single call was authorized to produce. Ruan et al. [76] analyze this via ToolEmu, an emulated environment revealing misuse patterns such as unauthorized data access, unintended side effects, and missing confirmation for irreversible actions, while R-Judge [104] shows substantial gaps in current models' ability to reject risky tool-calls. Real-world incidents confirm the severity: CVE-2025-53773 [64] in GitHub Copilot achieved remote code execution by letting the AI modify its own environment, escalate privileges, and run arbitrary code on the developer's machine, and related GitHub Copilot exploits induced the assistant to exfiltrate secrets from private repositories. Pedro et al. [70] document Prompt-to-SQL injection, where natural-language instructions are faithfully translated into malicious SQL, and Liu et al. [55] catalog RCE vulnerabilities in LLM-integrated applications, identifying tool interfaces as the primary attack surface. These attacks are countered by privilege control (Section 6.2), particularly Progent [83], which achieves 0% ASR by enforcing fine-grained tool policies in a deterministic runtime.

### 5.2.3 Memory Poisoning.

**Attacker: T2 (Data Poisoner), T3 (Tool Provider) | Properties violated: Integrity, Safety.** Memory poisoning targets the persistence mechanisms that distinguish agents from stateless chatbots. Unlike indirect injection, whose effect depends on malicious content being retrieved during the current execution, memory poisoning embeds adversarial content into the agent's long-term memory, where it persists across sessions and influences future behavior even after the original source is gone. Several mechanisms have been demonstrated. BadAgent [92] implants backdoors via adapter-based fine-tuning (AdaLoRA, QLoRA), needing only 20 percent poisoned data to exceed 90 percent ASR on web-navigation and shopping tasks; critically, these backdoors survive later legitimate fine-tuning, defeating standard data-centric mitigation. DemonAgent [112] uses dynamically encrypted multi-backdoor triggers that bypass advanced safety audits, reaching approximately 100 percent ASR with zero detection. AgentPoison [19] instead optimizes universal triggers so that poisoned demonstrations are retrieved from memory whenever an instruction contains the trigger—requiring no additional training and transferring across retrieval systems. At the application level, query-only memory injection [25] shows an attacker can poison memory simply by interacting normally: the agent stores attacker inputs as memories that later sessions retrieve, requiring no special access. Memory poisoning is particularly dangerous on four counts: it is *persistent*, remaining active across sessions until explicitly removed; *compounding*, since self-updating agents may store outputs influenced by corrupted memories as new entries; *stealthy*, as poisoned memories are largely indistinguishable from legitimate ones absent provenance tracking or integrity verification; and, most importantly, *undefended*—Section 6.4 identifies no published defense designed specifically to secure agent memory. This gap is among the most serious in the current defense landscape and is formalized as Open Problem 2 in Section 8.

### 5.2.4 RAG Poisoning.

**Attacker: T2 (Data Poisoner) | Properties violated: Integrity.** RAG poisoning targets the knowledge base from which the agent retrieves information, rather than the individual documents an agent happens to access (as in Section 5.2.1). PoisonedRAG [114] demonstrates corpus-level contamination where the attacker modifies a small number of documents in the external knowledge

base, causing the retriever to surface poisoned content for targeted queries. Adversarial embedding attacks manipulate the embedding space so that poisoned documents are retrieved even for benign queries [109]. This distinction from indirect injection is important: whereas indirect injection requires an attacker to control a specific document retrieved by the agent, RAG poisoning targets the retrieval infrastructure itself, thereby influencing all queries whose embedding representations match the poisoned entries. This makes RAG poisoning more persistent and harder to detect, though it requires deeper access to the knowledge base, implying a more capable attacker.

### 5.2.5 Tool supply-chain attacks.

**Attacker: T3 (Malicious Tool Provider) | Properties violated: All five properties potentially.** The rapid expansion of agent tool ecosystems—MCP servers, LangChain integrations, API connectors, and third-party plugins—introduces a supply-chain attack surface resembling npm or PyPI but with far weaker safeguards: no standardized certification or vetting governs agent tools, and any developer can publish an MCP server that agents adopt with little verification. LeechHijack [107] demonstrates covert resource exploitation through MCP plugins at up to 77 percent ASR while keeping overheads statistically indistinguishable from normal operation, rendering it practically invisible to monitoring. The Log-To-Leak framework [41] (Section 5.2.1) shows a malicious MCP logging tool exfiltrating all user-agent interactions while appearing legitimate. Tool-description manipulation is especially insidious: an attacker publishes a benign-looking tool, waits for installation, then alters the description to inject instructions—many MCP clients do not track description changes [29]. Real-world evidence includes the 2025 GitHub Copilot CVE series [64], where malicious instructions embedded in public repositories, READMEs, and notebooks triggered exfiltration of private-repo secrets when developers used Copilot. Ferrag et al. [29] document the Toxic Agent Flow exploit in GitHub’s MCP server and P2SQL injection via LangChain, the first systematic analysis of protocol-level vulnerabilities. Supply-chain defenses remain immature (Section 6.4), a critical gap formalized as Open Problem 3 in Section 8.

## 5.3 Layer 4 Attacks: Orchestration and Planning

Layer 4 attacks target the agent’s decision-making layer, including planning, privilege control, and multi-agent coordination. These threats are increasingly important as agents move from single-task assistants to coordinated, production-grade systems.

### 5.3.1 Multi-Agent Manipulation.

**Attacker: T4 (Compromised Agent) | Properties violated: Integrity, Safety/Alignment, Availability.** Multi-agent systems broaden the attack surface from a single agent to an interconnected ecosystem of communicating agents, where the inter-agent channel—trust boundary TB-4 (Section 3.1)—is a particularly critical vulnerability, since safety alignment is designed for human-to-AI rather than AI-to-AI trust. Lupinacci et al. [60] provide the most comprehensive study, testing three vectors against 17 models and revealing an alarming hierarchy: 41.2 percent of models succumb to direct prompt injection, 52.9 percent to RAG backdoor attacks, and a critical 82.4 percent to inter-agent communication. Most strikingly, models that resisted direct malicious commands from humans executed identical payloads when they arrived from a peer agent—demonstrating that safety training establishes a boundary for human-AI but not AI-AI interaction, a fundamental architectural gap. Other vectors compound this: Prompt Infection [52] propagates malicious instructions across agent boundaries at near-100 percent infection rates; CORBA [110] spreads contagious recursive denial-of-service through resource depletion, causing cascading failures that alignment alone cannot mitigate; and Agent-in-the-Middle (AiTM) attacks [36] achieve over 90 percent success by intercepting inter-agent messages, needing knowledge of only one agent’s incoming traffic.



Multi-agent trust frameworks remain early-stage (Section 6.4): proposals such as topology-aware anomaly detection (G-Safeguard [91]) and Byzantine-robust coordination [49] exist, but none has a production-ready implementation.

### 5.3.2 *Planning Subversion.*

**Attacker: T2 (Data Poisoner) | Properties violated: Integrity, Safety/Alignment .** Planning subversion attacks operate at Layer 4 by targeting the agent’s task decomposition and action selection. Agent Security Bench identifies Plan-of-Thought (PoT) backdoor attacks, in which hidden instructions in the system prompt induce unintended actions under specific triggers, yielding a 42.1% ASR [106]. A related threat, chain-of-thought hijacking, manipulates intermediate reasoning during multi-step execution, steering the agent off its intended plan without producing an explicit violation at any single step. Because individual actions appear harmless in isolation, such attacks evade step-level filters and instead require holistic plan-level verification—a capability largely absent in current agent systems.

### 5.3.3 *Privilege Escalation.*

**Attacker: T1 (Malicious End User), T4 (Compromised Agent) | Properties violated: Integrity .** Privilege escalation occurs when an attacker causes the agent to act beyond its intended authorization. The MAC framework [47] systematically demonstrates escalation bypasses against several published defenses, including SecAlign [17], IsolateGPT [98], and CaMeL [22]. Cross-tool privilege chains are especially dangerous: an agent induced to call a benign tool (read an email) may chain its output to a privileged one (extract an API key), then use that key to authorize a financial transfer. He et al. [35] identify a capability-vulnerability tradeoff—more capable generalist agents are more susceptible, since broader tool access creates more chaining opportunities. CVE-2025-53773 in GitHub Copilot illustrates the risk concretely: the system could modify its own environment configuration, effectively escalating its own privileges. Privilege control (Section 6.2), particularly Progent’s deterministic policy enforcement, is the primary defense against such attacks [64].

## 5.4 *Layer 1 Attacks: Infrastructure*

Layer 1 attacks target the hardware, network, and cloud infrastructure underlying the agent system. While many of these attacks are inherited from general systems security, agent architectures introduce specific amplifications.

### 5.4.1 *Side-Channel and Network Attacks.*

**Attacker: T5 (Network Adversary), T6 (Infrastructure Provider) | Properties violated: Confidentiality, Privacy .** Weiss et al. [95] demonstrate a token-length side-channel attack that infers the content of encrypted LLM API traffic by analyzing packet sizes, exploiting the fact that different tokens produce different-length encrypted payloads. This is particularly relevant for agents, where the content of tool-calls and their responses may reveal sensitive business logic or user data. Timing attacks on API calls can reveal which tools an agent is using and how long each operation takes, potentially disclosing the nature of the task being performed. At the infrastructure layer, honest-but-curious cloud providers like attacker T6 present a significant privacy risk, as they may observe GPU memory during inference, analyze activation patterns, and potentially recover sensitive training data through memorization-based probing. This risk becomes even more pronounced in split-inference deployments, where part of the model executes on a mobile device and the remaining portion runs in the cloud, thereby introducing an additional leakage surface through the transmission of intermediate activations over the network. These threats are addressed by infrastructure-level defenses (Section 6.3), including trusted execution environments, encrypted

inference protocols, and traffic padding techniques, although such protections typically incur substantial performance overhead.

## 5.5 Layer 5 Attacks: Output and User Interface

Layer 5 attacks target the agent's user interface and output channel. Most are downstream effects of earlier compromises: an injection entering through Layer 3 may corrupt reasoning at Layers 2 or 4 and finally appear at Layer 5 as misleading output, unauthorized action, or privacy leakage.

### 5.5.1 Data Exfiltration via Output Channels.

**Attacker: T2 (via indirect injection)/ Properties violated: Confidentiality, Privacy.** Data exfiltration occurs when the agent is manipulated into including sensitive information in its outputs or tool-call parameters. Alizadeh et al. [2] show that even simple injection attacks can leak personal data during task execution, and Green et al. [30] show that chain-of-thought traces expose private data—a risk for any system where reasoning is visible or logged. Shao et al. [82] provide PrivacyLens, a benchmark for privacy-norm violations in agent outputs, establishing a systematic basis for assessing this threat. Common strategies include encoding sensitive data in URL query parameters that the agent then "visits," embedding private information in email recipients or subject fields, and inserting sensitive content into tool-call parameters that may be logged or forwarded. Output-level defenses (Section 6.3)—PII detection, output moderation, and audit logging—offer partial mitigation.

## 5.6 Cross-Layer Attack Chains

The layer-based taxonomy in Sections 5.1–5.5 describes individual attack vectors, but many realistic agent compromises unfold as cross-layer chains. Table 4 summarizes three representative chains and shows how attacks propagate across the deployment stack. These chains show that agent attacks are often non-local: the security violation may not appear at the initial entry point but only after information crosses multiple layers and trust boundaries. In particular, a sequence of individually benign actions can become harmful in composition, such as reading a file, extracting a key, and using that key in a later tool call. This property makes purely per-action defenses insufficient and motivates the defense taxonomy in Section 6, where stronger approaches reason about cross-layer information flow, tool privileges, and plan-level behavior.

## 5.7 Attack Effectiveness Summary

Table 5 summarizes the attack taxonomy across target layer, attacker capability, measured effectiveness, evidence maturity, and defense availability. Three patterns emerge. First, the most severe agent-specific threats concentrate in Layers 3 and 4. These include indirect prompt injection exceeding 90% attack success through HTML accessibility tree injection [50], memory poisoning reaching approximately 100% attack success with zero detection [112], and multi-agent manipulation compromising 82.4% of tested models through inter-agent communication [60]. Second, the largest defense gaps align with these same threats: memory poisoning lacks dedicated defenses, tool supply-chain attacks lack certification mechanisms, and multi-agent trust remains immature. Third, the evidence base is uneven: Layer 2 attacks such as direct injection and jailbreaking are well studied but less specific to agentic systems, whereas Layer 3 and Layer 4 attacks are more agent-specific yet remain less systematically evaluated. These patterns motivate the defense taxonomy in Section 6 and the open problems formalized in Section 8.

Table 4. Representative cross-layer attack chains in LLM agent systems.

| Attack Chain                                       | Path                   | Mechanism                                                                                                                                                                          | Impact                                                                                                                                    |
|----------------------------------------------------|------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| Indirect injection to data exfiltration            | L3 → L2 → L4 → L3 → L5 | A T2 attacker plants malicious instructions in retrieved web content; the payload enters the flat token stream, hijacks planning, and triggers an unintended email-tool call [31]. | Private data is exfiltrated through an external output channel.                                                                           |
| Supply-chain attack to persistent memory poisoning | L3 → L3 → L4 → L5      | A T3 attacker provides a malicious tool or MCP server that appears legitimate but gradually injects poisoned outputs into long-term memory.                                        | Later sessions retrieve poisoned memories, causing incorrect recommendations or unsafe decisions.                                         |
| Multi-agent privilege escalation                   | L4 → L2 → L3 → L5      | A T4 compromised agent sends adversarial messages to a peer agent; the peer follows injected instructions because AI-to-AI trust is weakly enforced [60].                          | The attacker induces a sequence of privileged tool calls, such as reading files, extracting credentials, or authorizing external actions. |

Table 5. Comparative overview of attack classes across the deployment stack. ASR = Attack Success Rate (highest reported). Evidence maturity rated from ★ (emerging) to ★★★★★ (extensively studied). Defense availability indicates current state of countermeasures, with section references to Section 6.

| Attack Class                      | \$Ref  | Layer | Attacker     | Highest ASR                               | Evidence | Defense Status (§6 Ref)  |
|-----------------------------------|--------|-------|--------------|-------------------------------------------|----------|--------------------------|
| <b>Layer 2: Model Inference</b>   |        |       |              |                                           |          |                          |
| Direct Prompt Injection           | §5.1.1 | L2    | T1           | 72.7% [ASB]                               | ★★★★★    | Partial (§6.1.1, §6.1.3) |
| Jailbreaking                      | §5.1.2 | L2    | T1           | High (adaptive bypasses all defenses)     | ★★★★★    | Partial (§6.1.3)         |
| Model Extraction & Privacy        | §5.1.3 | L2    | T5, T6       | Demonstrated                              | ★★★★     | Partial (§6.3)           |
| <b>Layer 3: Tool &amp; Memory</b> |        |       |              |                                           |          |                          |
| Indirect Prompt Injection         | §5.2.1 | L3→L2 | T2, T3       | >90% (HTML injection)                     | ★★★★★    | Partial (§6.1.1–§6.1.4)  |
| Tool Abuse & Function Exploit     | §5.2.2 | L3    | T1, T2       | RCE demonstrated (CVE-2025-53773)         | ★★★★     | Yes (§6.2)               |
| Memory Poisoning                  | §5.2.3 | L3    | T2, T3       | >90% (BadAgent); ~100% (DemonAgent)       | ★★★★     | Warning: None            |
| RAG Poisoning                     | §5.2.4 | L3    | T2           | High (corpus-level)                       | ★★★      | Minimal (§6.4)           |
| supply-chain attacks              | §5.2.5 | L3    | T3           | 77% (LeechHijack)                         | ★★       | Warning: Minimal (§6.4)  |
| <b>Layer 4: Orchestration</b>     |        |       |              |                                           |          |                          |
| Multi-Agent Manipulation          | §5.3.1 | L4    | T4           | 82.4% (inter-agent)                       | ★★       | Warning: Immature (§6.4) |
| Planning Subversion               | §5.3.2 | L4    | T2           | 42.1% (PoT backdoor)                      | ★★       | Research (§6.1.4)        |
| Privilege Escalation              | §5.3.3 | L4    | T1, T4       | Demonstrated (bypasses CaMeL, IsolateGPT) | ★★       | Partial (§6.2)           |
| <b>Layer 1: Infrastructure</b>    |        |       |              |                                           |          |                          |
| Side-Channel Attacks              | §5.4.1 | L1    | T5, T6       | Content inferable from packet sizes       | ★★       | Research (§6.3)          |
| <b>Layer 5: Output</b>            |        |       |              |                                           |          |                          |
| Data Exfiltration                 | §5.5.1 | L5    | T2 (via IPI) | Demonstrated                              | ★★★      | Partial (§6.3)           |

## 6 Defense Taxonomy (Organized by Deployment Stack Layer)

Section 5 showed that LLM agents face threats across all five layers, concentrated at Layer 3 (tool and memory) and Layer 4 (orchestration). This section answers *RQ2: What defense mechanisms exist for each attack class, and to what extent are they deployable under real-world constraints of latency, cost, scalability, and usability?* We organize defenses by category rather than by layer, tracing a four-generation arc: from ad-hoc prompt engineering (G1) through external detection (G2) and model-level training (G3) to architectural redesign (G4). This shift from treating symptoms to targeting root causes parallels the evolution of SQL-injection defenses, which moved from ad hoc sanitization to principled mechanisms such as parameterized queries. Whereas prior surveys evaluate defenses mainly by attack-success reduction, we assess deployment readiness across eight dimensions, defined in Table 6 and applied to every defense below. The five security properties

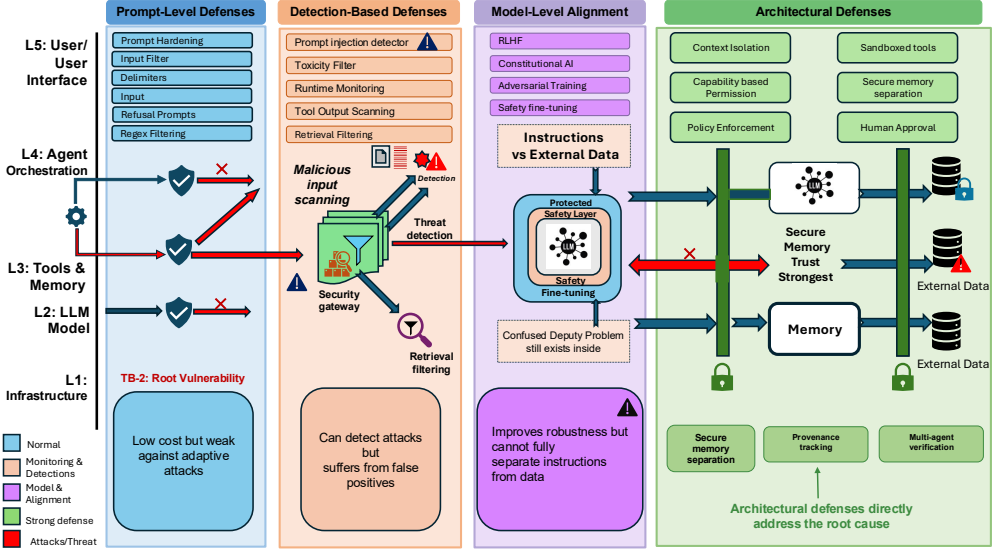


Fig. 4. Cross-layer defense taxonomy for LLM agentic systems.

Table 6. Deployment-readiness criteria used to evaluate defenses.

| Criterion           | Meaning                                                | Reported as                       |
|---------------------|--------------------------------------------------------|-----------------------------------|
| ASR Reduction       | Reduction in attack success after applying the defense | Percentage                        |
| Utility Impact      | Change in legitimate task completion under the defense | Percentage drop from baseline     |
| Latency Overhead    | Additional time introduced per agent action            | Milliseconds                      |
| Cost Overhead       | Additional compute or API cost                         | Multiplier or cost per 1K queries |
| False Positive Rate | Legitimate actions incorrectly blocked                 | Percentage                        |
| Deployability       | Access level required to implement the defense         | Provider / Framework / User       |
| Maturity            | Current implementation and deployment status           | Research / Prototype / Production |
| Adaptive Resistance | Robustness against adversaries aware of the defense    | Yes / Partial / No                |

from Section 4.2—Integrity, Confidentiality, Availability, Safety/Alignment, and Privacy—are the targets each defense aims to preserve, and for each we identify which Section 5 attacks it counters.

## 6.1 The Defense Evolution: Generation 1 through Generation 4

**6.1.1 Prompt-Level Defenses (Generation 1).** The earliest defenses mitigated prompt injection through changes to the prompt template alone, requiring no model or system changes. These Generation 1 approaches emerged in 2022–2023 as immediate responses to prompt injection. Prompt sandwiching repeats the system instructions after tool outputs and retrieved data to reassert intended behavior; on AgentDojo [23] it reduces ASR to 30.8 percent at only 3 percent utility loss—an improvement, but still nearly one in three attacks succeeding. Data delimiters wrap untrusted content in XML or JSON markers to separate data from instructions, but Willison [96] notes this offers no real security against an adaptive attacker who simply includes the closing

delimiter in the payload. Task Shield [48] instead keeps the agent on-task by pre-specifying expected actions, blunting injections that try to redirect behavior.

**Deployment assessment.** Generation 1 defenses are the most accessible—anyone with API access can deploy them at zero compute cost—but are trivially bypassed by adaptive adversaries and should never be the sole defense. Their value is as a low-cost defense-in-depth layer that catches unsophisticated attacks. Their limitations motivated external detection mechanisms.

**6.1.2 Detection-Based Defenses (Generation 2).** Generation 2 defenses insert an external check—a classifier or secondary LLM—between untrusted data and the primary model, identifying and removing adversarial content before it reaches the reasoning process. Early approaches used fine-tuned classifiers: ProtectAI’s DeBERTa model offers lightweight detection at moderate accuracy [46, 72], and DataSentinel [57] formulates detection game-theoretically, using a hidden known-answer instruction so that injected prompts which derail it reveal themselves. Meta’s Llama Prompt Guard 2 [45] ships within LlamaFirewall, a multi-component guardrail combining AlignmentCheck (a chain-of-thought auditor), PromptGuard (injection detection), and CodeShield (code safety) [20]—the most commercially mature G2 defense. LLM-based detectors inspect tool outputs with a secondary model, reaching 8 percent ASR on AgentDojo but at a 16 percent utility cost, frequently flagging legitimate outputs [23]; tool filtering by task context achieves similar ASR (7.5 percent) at similar cost. IPI-Guard instead analyzes tool dependency graphs, flagging anomalous call sequences [3]. The most significant development is PromptArmor [84], which overturns the assumption that prompting LLMs for detection is ineffective: by strategically prompting an off-the-shelf GPT-4o, it detects and removes injected prompts via fuzzy matching at below 1 percent false-positive, false-negative, and attack-success rates on AgentDojo while preserving utility—suggesting frontier LLMs’ detection capabilities were underestimated. Additional approaches include MELON [111], which compares agent behavior with and without suspicious data via masked re-execution, and InstruCoT [14], which fine-tunes chain-of-thought detection at a 90.9 percent rate.

**Deployment assessment.** Generation 2 defenses deploy with API access at moderate cost—classifiers add roughly 50–200 ms latency, LLM-based detectors roughly double inference cost—and PromptArmor reports the strongest performance (sub-1% ASR). Two limitations persist. First, detection happens only after the attack enters the pipeline, so a compromised detector (e.g., via cascading injection) can fail entirely. Second, robustness to adaptive adversaries is unestablished: PromptArmor is evaluated against known patterns, not attackers optimizing to evade it—a concern reinforced by evidence that adaptive attacks systematically bypass published jailbreak and injection defenses [63]. Detection thus addresses symptoms rather than causes, motivating model-level defenses.

**6.1.3 Model-Level Defenses (Generation 3).** Rather than detecting attacks externally, Generation 3 defenses modify the LLM’s weights during training to make the model itself resistant to manipulation. Foundational alignment techniques—RLHF [8, 67], Constitutional AI [9], and DPO [74]—form the baseline but were not designed for agent-context injection. SecAlign [17], the first model-level defense targeting prompt injection, uses preference optimization (an extension of DPO) to maintain intended behavior even when adversarial instructions appear in context, achieving near-zero ASR. However, against the adaptive RL-based AutoInject framework [18], SecAlign-70B still allows a 22% attack success rate, and PromptArmor’s evaluation reports degraded instruction-following—and thus poor utility—on AgentDojo even under benign conditions, a serious concern where task utility matters as much as robustness. Complementary approaches include Instruction Hierarchy [89], which trains the model to rank instructions by source (system over

user over tool outputs) and has been adopted by OpenAI, and StruQ [16], which separates user instructions from retrieved data.

**Deployment assessment.** Generation 3 defenses add zero runtime overhead, since protection lives in the weights, but are deployable only by model providers controlling the training pipeline—downstream API users cannot apply them. Their fundamental limit reflects the Section 4.3 root cause: all inputs still pass through the same attention mechanism, so as AutoInject’s 22% ASR shows, adaptive adversaries can still find inputs that circumvent any learned boundary. This motivates the shift to Generation 4.

**Differential privacy—a critical misconception.** DP-SGD and its successors protect training-data confidentiality during fine-tuning, but this is frequently conflated with inference-time protection. We emphasize that differential privacy provides zero defense against the inference-time attacks of Section 5: regardless of DP-SGD training, agents can still leak user data through tool-calls, embed PII in outputs, or expose memory contents. This matters because prior surveys often list DP-SGD as an agent defense without acknowledging the limitation.

**6.1.4 Architectural and System-Level Defenses (Generation 4).** Generation 4 defenses represent a paradigm shift and a central insight of this survey: rather than training the model to resist attacks (G3) or detecting them externally (G2), they redesign the system architecture so attacks are ineffective even if the model is compromised. The parallel to traditional security is instructive—from input sanitization to parameterized queries against SQL injection, from manual coding to memory-safe languages against buffer overflows, and from antivirus detection to sandboxed execution against malware. In each case the breakthrough came not from hardening the vulnerable component in isolation but from redesigning the surrounding system so its underlying vulnerability became far less consequential. Beurer-Kellner et al. [10] systematize this into design patterns—including plan-then-execute and context-minimization—that constrain agents so injected instructions cannot trigger consequential actions.

**CaMeL.** implements this principle through a dual-LLM architecture separating control flow from data flow [22]. A Privileged LLM generates the execution plan (control flow) from only the trusted user query and system prompt, never processing untrusted inputs, while a Quarantined LLM handles retrieved documents, tool outputs, and memory (data flow), extracting information but unable to influence tool invocation. A deterministic runtime then executes the plan, enforcing data-flow boundaries through capability-based security—so even a fully compromised Quarantined LLM cannot modify the plan. CaMeL achieves provable security on 77% of AgentDojo tasks versus 84% undefended, a 7% utility reduction. However, the MAC framework [47] later found an indirect-injection bypass in certain configurations, showing that even architectural defenses require continuous evaluation.

**RTBAS.** takes a complementary approach via runtime information-flow control, analogous to taint analysis in traditional software security [108]. Each data item is labeled trusted (from the user or system prompt) or untrusted (from tools, retrieved documents, or memory), and lightweight dependency screeners—implemented as LLM-as-judge modules or attention-based saliency models—track how these labels propagate during execution. When untrusted data influences a tool-call, the call is blocked; when private data would reach an output channel, the output is sanitized. RTBAS achieves 0% ASR against all targeted injection attacks in AgentDojo at only 2% utility loss—the strongest security–utility tradeoff among the defenses surveyed—and delivers near-oracle detection of both subtle and direct privacy leakage, addressing the threats in Section 5.5.1.

**Deployment assessment.** Generation 4 defenses achieve the strongest security, offering provable or near-provable guarantees that hold even against adaptive adversaries. However, they require fundamental architectural changes, making them deployable mainly by framework developers rather than end users of existing platforms. Both CaMeL and RTBAS remain at the research-to-prototype stage: open-source implementations exist, but no production deployments have been documented. The central challenge is scaling these architectures to real-world deployment without sacrificing the flexibility and generality that make agent systems valuable.

## 6.2 Access Control and Privilege Management

Regardless of whether an injection is detected or the model is compromised, restricting the tools that an agent may invoke, as well as the manner in which those tools may be invoked, helps contain the impact of a successful attack. This category functions independently of Generations 1 through 4 and therefore provides an additional layer of defense in depth.

**Progent.** , the strongest defense in this category, introduces a domain-specific language (DSL) for fine-grained tool policies—for example, allowing `send_email` only to contacts, or denying `execute_code` from untrusted sources [83]. A deterministic runtime enforces these policies at execution time, independently of the LLM, so unauthorized actions are blocked even if the LLM is fully compromised. Across AgentDojo, ASB, and AgentPoison, Progent achieves 0% ASR while preserving utility and speed. Its main limitation is that policies must be written manually, though LLMs can draft initial versions.

**IsolateGPT.** IsolateGPT provides execution isolation by separating environments per application, requiring user intervention for cross-application or irreversible actions [98]. The MAC framework [47] subsequently exposed a bypass through an anonymous Spoke component that lacks permission checks, illustrating the defense-attack co-evolution. TrustAgent implements pre-execution, during-execution, and post-execution safety checks based on agent-specific safety constitutions [42]. Prompt Flow Integrity prevents cross-tool privilege chains [51], the escalation pattern where a benign tool-call chains to a privileged one in Section 5.3.3.

**Deployment assessment.** Progent offers the most favorable profile in this category: roughly 10 ms latency for deterministic policy checking, no measurable utility loss, and strong effectiveness across all evaluated attacks, requiring framework-level integration but no model modification. Combining Progent for privilege control with CaMeL or RTBAS for architectural separation constitutes the most comprehensive defense stack identified in this survey.

## 6.3 Complementary Defense Mechanisms

**Runtime Monitoring and Human Confirmation.** Runtime monitoring defenses observe agent behavior during execution and intervene on anomalies or policy violations. GuardAgent uses an LLM-based safety orchestrator that validates proposed actions before execution, providing context-adaptive guardrails [100]; Agrail implements adaptive guardrails that evolve with agent behavior to handle novel situations [59]; and standard controls such as action allowlists, budget constraints, and rate limits offer simple but effective containment for known scenarios [110]. Human-in-the-loop confirmation merits special attention as the most widely deployed defense in production: commercial agents from OpenAI, Anthropic, and Microsoft require explicit user approval before high-risk actions such as sending emails, making payments, or deleting files. A human reviewer can block adversarial or unintended actions, a strong security benefit, but at substantial usability cost—it interrupts autonomous workflows and scales poorly to background or batch execution. The

challenge highlighted by CaMeL and RTBAS is therefore to achieve comparable security without human intervention.

**Infrastructure and Cryptographic Defenses.** These defenses protect Layer 1 against the attacks in Section 5.4. Trusted execution environments (TEEs) such as Intel SGX and ARM TrustZone provide hardware-isolated enclaves against curious infrastructure providers (T6), but current implementations cannot efficiently support full-scale LLM inference due to memory limits [87]. Homomorphic encryption offers strong privacy by computing on encrypted data, but its  $100\times$ – $10,000\times$  overhead [33] makes it impractical for interactive agents. Standard network mechanisms—TLS, certificate pinning, and API key rotation—are mature and mitigate network adversaries (T5), though not infrastructure-level attacks. Token-length padding and traffic shaping counter the side-channel attacks of Weiss et al. [95] but remain early-stage.

**Output and Privacy Defenses.** Output-level defenses form the final layer of protection by catching sensitive information before it leaves the agent at Layer 5. PII detection tools such as Microsoft Presidio and AWS Comprehend redact personal data prior to delivery, and content-moderation APIs classify outputs for safety violations. Privacy guardrails provide agent-specific protection frameworks [7], while PrivacyLens establishes evaluation criteria for privacy-norm violations in agent outputs [82]. However, these defenses address the symptom—leaked information in the output—rather than the cause, such as a Layer 3 injection that already compromised the agent; they are necessary for defense-in-depth but insufficient alone. Audit logging of tool invocations, memory accesses, and outputs supports post-hoc analysis and compliance, and the BELLS framework [26] benchmarks the effectiveness of such safeguards.

#### 6.4 Defense Gaps: Memory Security, supply-chain, and Multi-Agent Trust

**Memory and RAG Security.** Memory poisoning (Section 5.2.3) achieves attack success rates above 90%, yet no published defense specifically targets agent memory integrity—the most serious gap in the current landscape. Proposed but unimplemented directions include trusted write policies (only verified sources may write to long-term memory), memory provenance tracking (metadata recording who wrote each entry, when, and from where), and expiration policies that remove stale memories to limit persistent poisoning. For RAG security, input sanitization strips dangerous patterns from retrieved documents, SpotLight uses encoding to separate instructions from data [37], and retrieval verification checks content against query intent; all remain prototypes, fundamentally limited by the model’s inability to distinguish instructions from data (Section 4.3). Protocol security is similarly underdeveloped: the MCP and A2A specifications provide baseline mechanisms such as OAuth authentication, schema validation, transport encryption, agent identity verification, and capability negotiation, but neither offers a certification or vetting process for third-party tools—there is no app-store-style review layer for agent tools. We formalize this as Open Problem 3 in Section 8.

**Multi-agent trust frameworks.** Defenses against multi-agent manipulation (Section 5.3.1) remain at an early research stage. G-Safeguard applies graph-based anomaly detection to multi-agent network topologies, identifying suspicious communication patterns [91]. Jo and Park adapt Byzantine fault tolerance from distributed systems to multi-agent LLM coordination, providing robustness against compromised agents [49]. Kim et al. define seven security challenges for cross-domain multi-agent systems, establishing a research agenda [51]. However, no production-ready implementation exists for any multi-agent trust framework, and the fundamental AI-to-AI trust gap identified in Section 5.3.1 remains unaddressed.



Table 7. Defense effectiveness organized by deployment stack layer, mirroring the attack taxonomy in Table 5.

| Defense                                    | \$Ref   | Layer | Gen.      | Counters (\$5.x) | ASR After       | UA Loss | Maturity |
|--------------------------------------------|---------|-------|-----------|------------------|-----------------|---------|----------|
| <i>Layer 2: Model Inference Defenses</i>   |         |       |           |                  |                 |         |          |
| Prompt sandwiching                         | \$6.1.1 | L2    | G1        | \$5.1.1, \$5.2.1 | 30.8%           | ~3%     | Prod     |
| SecAlign                                   | \$6.1.3 | L2    | G3        | \$5.1.1, \$5.2.1 | 0% / 22%        | ~0%     | Prod*    |
| Instruction Hierarchy                      | \$6.1.3 | L2    | G3        | \$5.1.1          | Improved        | ~0%     | Prod*    |
| <i>Layer 3: Tool &amp; Memory Defenses</i> |         |       |           |                  |                 |         |          |
| PromptArmor                                | \$6.1.2 | L3→L2 | G2        | \$5.2.1          | <1%             | ~0%     | P        |
| LlamaFirewall                              | \$6.1.2 | L3→L2 | G2        | \$5.2.1, \$5.1.2 | Good            | Low     | Prod     |
| <b>CaMeL</b>                               | \$6.1.4 | L3→L2 | <b>G4</b> | \$5.2.1          | <b>Provable</b> | ~7%     | R        |
| <b>RTBAS</b>                               | \$6.1.4 | L3→L2 | <b>G4</b> | \$5.2.1, \$5.5.1 | <b>0%</b>       | ~2%     | R        |
| Progent                                    | \$6.2   | L3    | —         | \$5.2.2, \$5.3.3 | 0%              | ~0%     | P        |
| △ Memory defense                           | \$6.4   | L3    | —         | \$5.2.3          | <b>NONE</b>     | —       | —        |
| △ supply-chain cert.                       | \$6.4   | L3    | —         | \$5.2.5          | <b>NONE</b>     | —       | —        |
| <i>Layer 4: Orchestration Defenses</i>     |         |       |           |                  |                 |         |          |
| G-Safeguard                                | \$6.4   | L4    | —         | \$5.3.1          | Immature        | Unknown | R        |
| Byzantine-robust                           | \$6.4   | L4    | —         | \$5.3.1          | Immature        | Unknown | R        |
| Human confirmation                         | \$6.3   | L4    | —         | All              | Very high       | UX cost | Prod     |
| <i>Layer 1: Infrastructure Defenses</i>    |         |       |           |                  |                 |         |          |
| TEE (SGX/TrustZone)                        | \$6.3   | L1    | —         | \$5.4.1          | N/A             | 2–10×   | R        |
| TLS / cert pinning                         | \$6.3   | L1    | —         | \$5.4.1          | Standard        | ~0%     | Prod     |
| <i>Layer 5: Output Defenses</i>            |         |       |           |                  |                 |         |          |
| PII output filter                          | \$6.3   | L5    | —         | \$5.5.1          | Moderate        | FP risk | Prod     |
| Audit logging                              | \$6.3   | L5    | —         | \$5.5.1          | Post-hoc        | None    | Prod     |

Notes: Gen. = defense generation. ASR = residual attack success rate after defense is applied. UA = utility loss relative to undefended baseline. Maturity: R = Research, P = Prototype, Prod = Production. △ marks critical defense gaps. \*Deployable only by model providers. Bolded rows indicate Generation 4 architectural defenses with provable guarantees.

## 6.5 Defense Effectiveness Summary

Table 7 presents a condensed comparison of the most significant defenses identified in this survey, evaluated against the deployment-readiness criteria defined in Section 6.1. Three key findings emerge from this comparison. First, the progression from Generation 1 to Generation 4 reveals a clear tradeoff: attack success rates decline from 30.8% in Generation 1 to below 1% in Generation 2 and to 0% in Generation 4, while deployment accessibility simultaneously narrows from broadly available to anyone to limited primarily to framework developers. In other words, the most effective defenses are also the least accessible. Second, no existing defense simultaneously delivers strong security, minimal utility loss, and production level maturity. Among the available approaches, RTBAS comes closest to this objective, but it still remains at the research stage. Third, three critical attack classes, namely memory poisoning (Section 5.2.3), supply-chain attacks (Section 5.2.5), and multi-agent manipulation (Section 5.3.1), currently lack effective defenses. These unresolved gaps correspond directly to the open problems formalized in Section 8.

**Defense Novelty Classification.** Beyond the generational view (G1–G4), agent defenses can be organized by their *relationship to traditional security mechanisms*, clarifying which practitioners can deploy immediately, which require adapting established concepts, and which demand fundamentally new research.

**Tier 1: Conventional defenses directly applicable to agent systems.** These include role- and attribute-based access control (RBAC, ABAC), input validation, network security (TLS, mutual authentication, API key management), and logging with anomaly detection. These map to Layer 1 and

Layer 5 (Section 6.3) and deploy on existing infrastructure, but protect only the periphery—network transport, API endpoints, output channels—without addressing the confused deputy vulnerability at TB-2 (Section 4.3).

**Tier 2: Adapted conventional defenses requiring extension for agent contexts.** Established concepts whose translation is non-trivial because natural language breaks their original assumptions. Process sandboxing becomes semantic-level prompt and tool sandboxing; least privilege becomes capability-scoped tool invocation, as in Progent [83]; network policy enforcement becomes semantic-aware policy engines; supply-chain security becomes tool and plugin certification; and taint tracking becomes prompt and data provenance tracking, as in RTBAS [108]. These map to TB-2 and TB-3, where traditional assumptions break down most acutely.

**Tier 3: Novel agent-specific defenses without traditional analogues.** These failure modes from autonomy and multi-agent interaction—instruction-data separation through dual-model architectures such as CaMeL [22], semantic firewalls and intent validation, multi-agent trust protocols (Section 6.4), memory integrity verification via cryptographic signatures and provenance graphs (Section 6.4, open), and plan-level verification of action sequences. These mark the research frontier.

Overall, the landscape is less barren than raw gap counts suggest: practitioners can deploy Tier 1 as a baseline and prioritize Tier 2 for their trust-boundary exposure while the community develops Tier 3. The most pressing need is Tier 2—least privilege, taint tracking, and supply-chain verification have solid theoretical foundations, but adapting them to a setting where inputs are natural language, boundaries are semantic, and execution is probabilistic remains largely unsolved.

## 7 Cross-Cutting Analysis

Sections 5 and 6 analyze attacks and defenses individually. This section synthesizes three cross-cutting patterns: the tradeoff between security and utility, the need for composable defenses, and the limitations of current evaluation practice.

### 7.1 Security–Utility Tradeoff

The defense landscape reveals a persistent security–utility tradeoff, which Table 7 makes explicit. Lightweight defenses generally preserve agent flexibility but provide limited robustness, while stronger defenses often introduce latency, false positives, architectural constraints, or usability costs. For example, prompt-level defenses are easy to deploy but brittle against adaptive attacks; detection-based systems such as PromptArmor improve protection but add runtime overhead; model-level alignment can reduce template-based attacks but remains vulnerable to adaptive jailbreaks; and architectural defenses such as CaMeL and RTBAS provide stronger guarantees but require redesigning the agent execution pipeline. As a result, defense selection is deployment-specific: low-risk customer-service agents may prioritize responsiveness, whereas financial or code-execution agents may justify stricter enforcement, human confirmation, or architectural isolation. The deployment-readiness criteria in Section 6.1 provide a systematic basis for making this tradeoff.

### 7.2 Defense Composability

The cross-layer attack chains in Section 5.6 show that no single defense layer is sufficient. Effective protection requires defense-in-depth: detection mechanisms can filter suspicious inputs before they reach the model, architectural defenses can separate control flow from untrusted data when detection fails [22], and privilege-control mechanisms such as Progent can constrain tool invocation even after model reasoning has been influenced [83]. However, composability introduces its own risks. Stacked defenses may accumulate false positives, increase latency, and reduce agent utility,

Table 8. Open problems in LLM agent security.

| Open Problem                                 | Core Gap                                              | Research Direction                                               |
|----------------------------------------------|-------------------------------------------------------|------------------------------------------------------------------|
| OP1: Production-ready architectural defenses | Strong research prototypes lack production validation | Scalable enforcement, selective isolation, deployment evaluation |
| OP2: Agent memory security                   | Persistent poisoned memories lack dedicated defenses  | Provenance, integrity checks, memory evolution monitoring        |
| OP3: Tool ecosystem certification            | Third-party tools lack vetting or attestation         | Tool certification, behavioral monitoring, security metadata     |
| OP4: Inference-time privacy                  | Training-time DP does not prevent runtime leakage     | Output sanitization, private tool use, reasoning-trace privacy   |
| OP5: Multi-agent trust                       | AI-to-AI trust remains weakly modeled and defended    | Agent identity, message authentication, formal trust models      |
| OP6: Standardized evaluation                 | Benchmarks use incompatible scopes and metrics        | Unified cross-layer benchmarks with deployment criteria          |

and no existing benchmark systematically evaluates the combined effectiveness and overhead of multi-defense stacks. A further challenge is enforcement placement. Traditional architectures separate Policy Decision Points (PDPs), which evaluate whether an action is permitted, from Policy Enforcement Points (PEPs), which block unauthorized actions at runtime. In agent systems this separation is complicated by multi-step execution: a PEP at Layer 3 can block an individual tool-call but cannot judge whether a sequence of individually benign calls is collectively unsafe—reading an email, extracting a key from it, then using that key to authorize a transfer. This motivates future work on plan-level PDPs, cross-layer PEP placement, and enforcement architectures that incorporate multi-step context, as reflected in Open Problems 1 and 6. Multi-agent systems introduce an additional composability challenge because defenses must also operate across agent boundaries. Early work such as G-Safeguard and Byzantine-robust decentralized coordination begins to address this issue [49, 91], but these approaches remain preliminary and have not yet been integrated with broader agent defense stacks.

### 7.3 Evaluation Limitations

Current evaluation remains fragmented. Benchmarks such as AgentDojo [23], WASP [27], ASB [106], ToolEmu [76], R-Judge [104], and PrivacyLens [82] each cover important parts of the attack surface, but they differ in scope, tasks, metrics, and assumptions. No benchmark currently evaluates all five deployment layers while jointly measuring attack success, utility impact, latency, cost, false positives, and adaptive robustness. This makes cross-paper comparison difficult and limits the field’s ability to determine which defenses are deployable in practice. A unified cross-layer benchmark with deployment-readiness metrics is therefore necessary for mature evaluation of LLM-agent security.

## 8 Open Challenges and Future Directions

The attack and defense analyses in Sections 5 and 6 reveal six open problems that currently limit secure deployment of LLM-based agents. Table 8 summarizes these gaps, their causes, and promising research directions.

**OP1: Bridging Architectural Defenses to Production.** Generation 4 defenses such as CaMeL [22] and RTBAS [108] provide the strongest guarantees, with RTBAS reporting a 0% attack success rate and 2% utility loss. However, these systems remain research prototypes: CaMeL constrains dynamic planning, while RTBAS has not yet been evaluated at production scale. Future work should validate architectural defenses under realistic tool-call volume, concurrency, latency, and

deployment constraints, while exploring hybrid enforcement that applies strict controls only to high-risk actions.

**OP2: Agent Memory Security.** Memory poisoning is one of the largest attack-defense gaps: BadAgent reports attack success rates above 90% [92], and DemonAgent reaches approximately 100% success with zero detection [112]. Existing defenses do not directly address persistent, self-updating memory stores that can carry poisoned content across sessions. Future research should develop memory provenance, write-time authentication, retrieval-time integrity verification, and anomaly detection over memory evolution.

**OP3: Tool Ecosystem Certification.** The agent tool ecosystem, including MCP servers, plugins, and API connectors, lacks mature certification or vetting mechanisms. This gap enables attacks such as LeechHijack, which achieves 77% ASR through apparently legitimate MCP plugins [107], and tool-description manipulation that changes agent behavior after installation [29]. Future work should develop lightweight tool attestation, runtime behavioral monitoring, and standardized security metadata analogous to software bill of materials practices.

**OP4: Inference-Time Privacy.** As established in Section 6.1.3, training-time privacy techniques such as DP-SGD do not prevent inference-time leakage through reasoning traces, tool-call parameters, or output channels. Agents may expose private information through chain-of-thought reasoning [30] and other output channels (Section 5.5.1), even when the final response appears sanitized. Future work should focus on inference-time privacy mechanisms, including utility-preserving output sanitization, private tool invocation protocols, and formal privacy guarantees for agent reasoning traces.

**OP5: Multi-Agent Trust.** Multi-agent systems expose a major gap between human-to-AI and AI-to-AI trust: 82.4% of models are compromised through inter-agent communication, compared with 41.2% through direct injection [60]. Existing defenses remain early-stage, including G-Safeguard [91] and Byzantine-robust coordination [49]. Future work should develop cryptographic agent identity, authenticated inter-agent messaging, and formal trust models that preserve collaboration efficiency.

**OP6: Standardized Evaluation.** Current benchmarks, including AgentDojo [23], ASB [106], InjecAgent [105], and ToolEmu [76], evaluate different attack subsets with incompatible metrics. No benchmark jointly covers all five deployment layers while also measuring deployment-readiness criteria such as latency, cost, false positives, and utility impact. A unified evaluation framework is needed to compare defenses under realistic, cross-layer, and deployment-oriented conditions.

Together, these open problems show that the most severe agent-specific risks remain insufficiently defended. Progress will require architectural enforcement, trustworthy tool and memory infrastructure, multi-agent trust mechanisms, and standardized evaluation that reflects real deployment constraints.

## 9 Conclusion

This survey presented a deployment-centric framework for analyzing security in LLM-based agent systems, organizing attacks, defenses, and open challenges around a five-layer agent stack.

Our analysis leads to three main findings. First, attacks against LLM agents are inherently cross-layer: adversarial inputs can enter through tools, memory, user interfaces, or inter-agent communication, then propagate through model inference, planning, tool invocation, and output generation. Second, the defense landscape is shifting from prompt-level safeguards and model-level alignment toward architectural mechanisms that reduce the consequences of model compromise, including separation of control and data flow and deterministic privilege enforcement. Third, the

most agent-specific risks remain under-defended: memory poisoning, tool supply-chain compromise, and multi-agent manipulation expose critical gaps that current defenses do not yet address with sufficient maturity or deployability.

The central lesson is that agent security cannot be reduced to LLM safety alone. The capabilities that make agents useful—tool invocation, persistent memory, autonomous planning, and multi-agent collaboration—also create new security boundaries and failure modes. Securing future agentic systems will require cross-stack defenses that combine architectural isolation, runtime enforcement, trustworthy tool ecosystems, memory integrity, and standardized evaluation under realistic deployment constraints.

## References

- [1] Christopher M. Ackerman and Nina Panickssery. 2025. Mitigating Many-Shot Jailbreaking. arXiv:2504.09604 [cs.LG] <https://arxiv.org/abs/2504.09604>
- [2] Meysam Alizadeh, Zeynab Samei, Daria Stetsenko, and Fabrizio Gilardi. 2025. Simple Prompt Injection Attacks Can Leak Personal Data Observed by LLM Agents During Task Execution. arXiv:2506.01055 [cs.CR] <https://arxiv.org/abs/2506.01055>
- [3] Hengyu An, Jinghui Zhang, Tianyu Du, Chunyi Zhou, Qingming Li, Tao Lin, and Shouling Ji. 2025. IPIGuard: A Novel Tool Dependency Graph-Based Defense Against Indirect Prompt Injection in LLM Agents. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, Suzhou, China, 1023–1039. doi:10.18653/v1/2025.emnlp-main.53
- [4] Maksym Andriushchenko, Francesco Croce, and Nicolas Flammarion. 2025. Jailbreaking Leading Safety-Aligned LLMs with Simple Adaptive Attacks. In *The Thirteenth International Conference on Learning Representations*. <https://openreview.net/forum?id=hXA8wqRdyV>
- [5] Cem Anil, Esin Durmus, Nina Panickssery, Mrinank Sharma, Joe Benton, Sandipan Kundu, Joshua Batson, Meg Tong, Jesse Mu, Daniel Ford, Francesco Mosconi, Rajashree Agrawal, Rylan Schaeffer, Naomi Bashkansky, Samuel Svenningsen, Mike Lambert, Ansh Radhakrishnan, Carson Denison, Evan J Hubinger, Yuntao Bai, Trenton Bricken, Timothy Maxwell, Nicholas Schiefer, James Sully, Alex Tamkin, Tamera Lanhan, Karina Nguyen, Tomasz Korbak, Jared Kaplan, Deep Ganguli, Samuel R. Bowman, Ethan Perez, Roger Baker Grosse, and David Duvenaud. 2024. Many-shot Jailbreaking. In *Proceedings of the 38th International Conference on Neural Information Processing Systems (Vancouver, BC, Canada) (NIPS '24)*. Curran Associates Inc., Red Hook, NY, USA, Article 4121, 47 pages.
- [6] Anthropic. 2024. Introducing the Model Context Protocol. <https://www.anthropic.com/news/model-context-protocol>. Accessed: 2026-06-12.
- [7] Shubhi Asthana, Bing Zhang, Ruchi Mahindru, Chad DeLuca, Anna Lisa Gentile, and Sandeep Gopisetty. 2025. Deploying Privacy Guardrails for LLMs: A Comparative Analysis of Real-World Applications. arXiv:2501.12456 [cs.CR] <https://arxiv.org/abs/2501.12456>
- [8] Yuntao Bai, Andy Jones, Kamal Ndousse, Amanda Askell, Anna Chen, Nova DasSarma, Dawn Drain, Stanislav Fort, Deep Ganguli, Tom Henighan, Nicholas Joseph, Saurav Kadavath, Jackson Kernion, Tom Conerly, Sheer El-Showk, Nelson Elhage, Zac Hatfield-Dodds, Danny Hernandez, Tristan Hume, Scott Johnston, Shauna Kravec, Liane Lovitt, Neel Nanda, Catherine Olsson, Dario Amodei, Tom Brown, Jack Clark, Sam McCandlish, Chris Olah, Ben Mann, and Jared Kaplan. 2022. Training a Helpful and Harmless Assistant with Reinforcement Learning from Human Feedback. arXiv:2204.05862 [cs.CL] <https://arxiv.org/abs/2204.05862>
- [9] Yuntao Bai, Saurav Kadavath, Sandipan Kundu, Amanda Askell, Jackson Kernion, Andy Jones, Anna Chen, Anna Goldie, Azalia Mirhoseini, Cameron McKinnon, Carol Chen, Catherine Olsson, Christopher Olah, Danny Hernandez, Dawn Drain, Deep Ganguli, Dustin Li, Eli Tran-Johnson, Ethan Perez, Jamie Kerr, Jared Mueller, Jeffrey Ladish, Joshua Landau, Kamal Ndousse, Kamile Lukosuite, Liane Lovitt, Michael Sellitto, Nelson Elhage, Nicholas Schiefer, Noemi Mercado, Nova DasSarma, Robert Lasenby, Robin Larson, Sam Ringer, Scott Johnston, Shauna Kravec, Sheer El Showk, Stanislav Fort, Tamera Lanham, Timothy Telleen-Lawton, Tom Conerly, Tom Henighan, Tristan Hume, Samuel R. Bowman, Zac Hatfield-Dodds, Ben Mann, Dario Amodei, Nicholas Joseph, Sam McCandlish, Tom Brown, and Jared Kaplan. 2022. Constitutional AI: Harmlessness from AI Feedback. arXiv:2212.08073 [cs.CL] <https://arxiv.org/abs/2212.08073>
- [10] Luca Beurer-Kellner, Beat Buesser, Ana-Maria Crețu, Edoardo Debenedetti, Daniel Dobos, Daniel Fabian, Marc Fischer, David Froelicher, Kathrin Grosse, Daniel Naef, Ezinwanne Ozoani, Andrew Paverd, Florian Tramèr, and Václav Volhejn. 2025. Design Patterns for Securing LLM Agents against Prompt Injections. arXiv:2506.08837 [cs.LG] <https://arxiv.org/abs/2506.08837>
- [11] Nicholas Carlini, Daphne Ippolito, Matthew Jagielski, Katherine Lee, Florian Tramèr, and Chiyuan Zhang. 2023. Quantifying Memorization Across Neural Language Models. In *The Eleventh International Conference on Learning*

*Representations*. [https://openreview.net/forum?id=TatRHT\\_1cK](https://openreview.net/forum?id=TatRHT_1cK)

- [12] Nicholas Carlini, Florian Tramèr, Eric Wallace, Matthew Jagielski, Ariel Herbert-Voss, Katherine Lee, Adam Roberts, Tom Brown, Dawn Song, Úlfar Erlingsson, Alina Oprea, and Colin Raffel. 2021. Extracting Training Data from Large Language Models. In *30th USENIX Security Symposium (USENIX Security 21)*. USENIX Association, 2633–2650. <https://www.usenix.org/conference/usenixsecurity21/presentation/carlini-extracting>
- [13] Hwan Chang, Yonghyun Jun, and Hwanhee Lee. 2026. ChatInject: Abusing Chat Templates for Prompt Injection in LLM Agents. In *The Fourteenth International Conference on Learning Representations*. <https://openreview.net/forum?id=WVhgFSKnIL>
- [14] Zhiyuan Chang, Mingyang Li, Yuekai Huang, Ziyou Jiang, Xiaojun Jia, Qian Xiong, Junjie Wang, Zhaoyang Li, and Qing Wang. 2026. Know Thy Enemy: Securing LLMs Against Prompt Injection via Diverse Data Synthesis and Instruction-Level Chain-of-Thought Learning. arXiv:2601.04666 [cs.AI] <https://arxiv.org/abs/2601.04666>
- [15] Harrison Chase. 2022. LangChain: Building applications with LLMs through composability. <https://github.com/langchain-ai/langchain>. Accessed: 2026-06-12.
- [16] Sizhe Chen, Julien Piet, Chawin Sitawarin, and David Wagner. 2025. StruQ: defending against prompt injection with structured queries. In *Proceedings of the 34th USENIX Conference on Security Symposium (Seattle, WA, USA) (SEC '25)*. USENIX Association, USA, Article 123, 18 pages.
- [17] Sizhe Chen, Arman Zharmagambetov, Saeed Mahloujifar, Kamalika Chaudhuri, David Wagner, and Chuan Guo. 2025. SecAlign: Defending Against Prompt Injection with Preference Optimization. In *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security (Taipei, Taiwan) (CCS '25)*. Association for Computing Machinery, New York, NY, USA, 2833–2847. doi:10.1145/3719027.3744836
- [18] Xin Chen, Jie Zhang, and Florian Tramèr. 2026. Learning to Inject: Automated Prompt Injection via Reinforcement Learning. arXiv:2602.05746 [cs.LG] <https://arxiv.org/abs/2602.05746>
- [19] Zhaorun Chen, Zhen Xiang, Chaowei Xiao, Dawn Song, and Bo Li. 2024. AgentPoison: Red-teaming LLM Agents via Poisoning Memory or Knowledge Bases. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*. <https://openreview.net/forum?id=Y841BRW9rY>
- [20] Sahana Chennabasappa, Cyrus Nikolaidis, Daniel Song, David Molnar, Stephanie Ding, Shengye Wan, Spencer Whitman, Lauren Deason, Nicholas Doucette, Abraham Montilla, Alekhya Gampa, Beto de Paola, Dominik Gabi, James Crnkovich, Jean-Christophe Testud, Kat He, Rashnil Chaturvedi, Wu Zhou, and Joshua Saxe. 2025. LlamaFirewall: An open source guardrail system for building secure AI agents. arXiv:2505.03574 [cs.CR] <https://arxiv.org/abs/2505.03574>
- [21] Badhan Chandra Das, M. Hadi Amini, and Yanzhao Wu. 2025. Security and Privacy Challenges of Large Language Models: A Survey. *ACM Comput. Surv.* 57, 6, Article 152 (Feb. 2025), 39 pages. doi:10.1145/3712001
- [22] Edoardo Debenedetti, Ilia Shumailov, Tianqi Fan, Jamie Hayes, Nicholas Carlini, Daniel Fabian, Christoph Kern, Chongyang Shi, Andreas Terzis, and Florian Tramèr. 2025. Defeating Prompt Injections by Design. arXiv:2503.18813 [cs.CR] <https://arxiv.org/abs/2503.18813>
- [23] Edoardo Debenedetti, Jie Zhang, Mislav Balunovic, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. 2024. AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents. In *Advances in Neural Information Processing Systems*, A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang (Eds.), Vol. 37. Curran Associates, Inc., 82895–82920. doi:10.52202/079017-2636
- [24] Zehang Deng, Yongjian Guo, Changzhou Han, Wanlun Ma, Junwu Xiong, Sheng Wen, and Yang Xiang. 2025. AI Agents Under Threat: A Survey of Key Security Challenges and Future Pathways. *ACM Comput. Surv.* 57, 7, Article 182 (Feb. 2025), 36 pages. doi:10.1145/3716628
- [25] Shen Dong, Shaochen Xu, Pengfei He, Yige Li, Jiliang Tang, Tianming Liu, Hui Liu, and Zhen Xiang. 2026. Memory Injection Attacks on LLM Agents via Query-Only Interaction. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*. <https://openreview.net/forum?id=QINnsnppv8>
- [26] Diego Dorn, Alexandre Variengien, Charbel-Raphael Segerie, and Vincent Corruble. 2024. BELLS: A Framework Towards Future Proof Benchmarks for the Evaluation of LLM Safeguards. In *ICML 2024 Next Generation of AI Safety Workshop*. <https://openreview.net/forum?id=5RlrE11t8c>
- [27] Ivan Evtimov, Arman Zharmagambetov, Aaron Grattafiori, Chuan Guo, and Kamalika Chaudhuri. 2025. WASP: Benchmarking Web Agent Security Against Prompt Injection Attacks. arXiv:2504.18575. <https://arxiv.org/abs/2504.18575>
- [28] Mohammad Fasha, Faisal Abul Rub, Nasim Matar, Bilal Sowam, Mohammad Al Khaldy, and Hussam Barham. 2024. Mitigating the OWASP Top 10 For Large Language Models Applications using Intelligent Agents. In *2024 2nd International Conference on Cyber Resilience (ICCR)*. IEEE, 1–9. doi:10.1109/ICCR61006.2024.10532874
- [29] Mohamed Amine Ferrag, Norbert Tihanyi, Djallel Hamouda, Leandros Maglaras, Abderrahmane Lakas, and Merouane Debbah. 2025. From Prompt Injections to Protocol Exploits: Threats in LLM-Powered AI Agents Workflows. arXiv:2506.23260 [cs.CR] doi:10.1016/j.ict.2025.12.001

- [30] Tommaso Green, Martin Gubri, Haritz Puerto, Sangdoo Yun, and Seong Joon Oh. 2025. Leaky Thoughts: Large Reasoning Models Are Not Private Thinkers. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, Christos Christodoulopoulos, Tanmoy Chakraborty, Carolyn Rose, and Violet Peng (Eds.). Association for Computational Linguistics, Suzhou, China, 26507–26529. doi:10.18653/v1/2025.emnlp-main.1347
- [31] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. 2023. Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security* (Copenhagen, Denmark) (AISec '23). Association for Computing Machinery, New York, NY, USA, 79–90. doi:10.1145/3605764.3623985
- [32] Yongjian Guo, Puzhuo Liu, Wanlun Ma, Zehang Deng, Xiaogang Zhu, Peng Di, Xi Xiao, and Sheng Wen. 2026. MCPXKIT: the Unified Toolkit for Analyzing Model Context Protocol Security. *IEEE Transactions on Dependable and Secure Computing* (2026), 1–16. doi:10.1109/TDSC.2026.3695553
- [33] Meng Hao, Hongwei Li, Hanxiao Chen, Pengzhi Xing, Guowen Xu, and Tianwei Zhang. 2022. Iron: private inference on transformers. In *Proceedings of the 36th International Conference on Neural Information Processing Systems* (New Orleans, LA, USA) (NIPS '22). Curran Associates Inc., Red Hook, NY, USA, Article 1143, 14 pages.
- [34] Norm Hardy. 1988. The Confused Deputy: (or why capabilities might have been invented). *SIGOPS Oper. Syst. Rev.* 22, 4 (Oct. 1988), 36–38. doi:10.1145/54289.871709
- [35] Feng He, Tianqing Zhu, Dayong Ye, Bo Liu, Wanlei Zhou, and Philip S. Yu. 2025. The Emerged Security and Privacy of LLM Agent: A Survey with Case Studies. *ACM Comput. Surv.* 58, 6, Article 162 (Dec. 2025), 36 pages. doi:10.1145/3773080
- [36] Pengfei He, Yuping Lin, Shen Dong, Han Xu, Yue Xing, and Hui Liu. 2025. Red-Teaming LLM Multi-Agent Systems via Communication Attacks. In *Findings of the Association for Computational Linguistics: ACL 2025*. Association for Computational Linguistics, Vienna, Austria, 6726–6747. doi:10.18653/v1/2025.findings-acl.349
- [37] Keegan Hines, Gary Lopez, Matthew Hall, Federico Zarfati, Yonatan Zunger, and Emre Kiciman. 2024. Defending Against Indirect Prompt Injection Attacks With Spotlighting. arXiv:2403.14720. <https://arxiv.org/abs/2403.14720>
- [38] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiwu Zheng, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, zili wang, Steven Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. 2024. MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework. In *International Conference on Learning Representations*, B. Kim, Y. Yue, S. Chaudhuri, K. Fragkiadaki, M. Khan, and Y. Sun (Eds.), Vol. 2024. 23247–23275. [https://proceedings.iclr.cc/paper\\_files/paper/2024/file/6507b115562bb0a305f1958ccc87355a-Paper-Conference.pdf](https://proceedings.iclr.cc/paper_files/paper/2024/file/6507b115562bb0a305f1958ccc87355a-Paper-Conference.pdf)
- [39] Xinyi Hou, Yanjie Zhao, Shenao Wang, and Haoyu Wang. 2026. Model Context Protocol (MCP): Landscape, Security Threats, and Future Research Directions. *ACM Trans. Softw. Eng. Methodol.* (Feb. 2026). doi:10.1145/3796519
- [40] Siyuan Hu, Mingyu Ouyang, Difei Gao, and Mike Zheng Shou. 2024. The Dawn of GUI Agent: A Preliminary Case Study with Claude 3.5 Computer Use. arXiv:2411.10323 [cs.AI] <https://arxiv.org/abs/2411.10323>
- [41] Yuepeng Hu, Chongyu Fan, Sirat Samyoun, and Jian Du. 2026. Log-To-Leak: Prompt Injection Attacks on Tool-Using LLM Agents via Model Context Protocol. <https://openreview.net/forum?id=UVgbFuXPaO>
- [42] Wenyue Hua, Xianjun Yang, Mingyu Jin, Zelong Li, Wei Cheng, Ruixiang Tang, and Yongfeng Zhang. 2024. TrustAgent: Towards Safe and Trustworthy LLM-based Agents. In *Findings of the Association for Computational Linguistics: EMNLP 2024*. Association for Computational Linguistics, Miami, Florida, USA, 10000–10016. doi:10.18653/v1/2024.findings-emnlp.585
- [43] Ken Huang. 2025. Agentic AI Threat Modeling Framework: MAESTRO. <https://cloudsecurityalliance.org/blog/2025/02/06/agentic-ai-threat-modeling-framework-maestro>. Cloud Security Alliance. Accessed: 2026-05-21.
- [44] Bo Hui, Haolin Yuan, Neil Gong, Philippe Burlina, and Yinzhi Cao. 2024. PLeak: Prompt Leaking Attacks against Large Language Model Applications. In *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security (CCS '24)*. ACM, 3600–3614. doi:10.1145/3658644.3670370
- [45] Hakan Inan, Kartikeya Upasani, Jianfeng Chi, Rashi Rungta, Krithika Iyer, Yuning Mao, Michael Tontchev, Qing Hu, Brian Fuller, Davide Testuggine, and Madian Khabsa. 2023. Llama Guard: LLM-based Input-Output Safeguard for Human-AI Conversations. arXiv:2312.06674. <https://arxiv.org/abs/2312.06674>
- [46] Yi Ji, Runzhi Li, and Baolei Mao. 2026. Detection Method for Prompt Injection by Integrating Pre-trained Model and Heuristic Feature Engineering. In *Knowledge Science, Engineering and Management*, Tianqing Zhu, Wanlei Zhou, and Congcong Zhu (Eds.). Springer Nature Singapore, Singapore, 66–73.
- [47] Zimo Ji, Daoquan Wu, Wenjuan Jiang, Pingchuan Ma, Zongjie Li, Yudong Gao, Shuai Wang, and Yingjiu Li. 2026. Taming Various Privilege Escalation in LLM-Based Agent Systems: A Mandatory Access Control Framework. arXiv:2601.11893 [cs.CR] <https://arxiv.org/abs/2601.11893>
- [48] Feiran Jia, Tong Wu, Xin Qin, and Anna Squicciarini. 2025. The Task Shield: Enforcing Task Alignment to Defend Against Indirect Prompt Injection in LLM Agents. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 29680–29697. doi:10.18653/v1/2025.acl-long.1435

- [49] Yongrae Jo and Chanik Park. 2025. Byzantine-Robust Decentralized Coordination of LLM Agents. arXiv:2507.14928 [cs.DC] <https://arxiv.org/abs/2507.14928>
- [50] Sam Johnson, Viet Pham, and Thai Le. 2025. Manipulating LLM Web Agents with Indirect Prompt Injection Attack via HTML Accessibility Tree. arXiv:2507.14799 [cs.CR] <https://arxiv.org/abs/2507.14799>
- [51] Juhee Kim, Woohyuk Choi, and Byoungyoung Lee. 2025. Prompt Flow Integrity to Prevent Privilege Escalation in LLM Agents. arXiv:2503.15547 [cs.CR] <https://arxiv.org/abs/2503.15547>
- [52] Donghyun Lee, Mo Tiwari, and Brando Miranda. 2026. Prompt Infection: LLM-to-LLM Prompt Injection within Multi-agent Systems. In *Computer Security. ESORICS 2025 International Workshops*. Springer Nature Switzerland, Cham, 511–520. doi:10.1007/978-3-032-16092-8\_28
- [53] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Proceedings of the 34th International Conference on Neural Information Processing Systems* (Vancouver, BC, Canada) (NIPS '20). Curran Associates Inc., Red Hook, NY, USA, Article 793, 16 pages.
- [54] Ang Li, Yin Zhou, Vethavikashini Chithra Raghuram, Tom Goldstein, and Micah Goldblum. 2025. Commercial LLM Agents Are Already Vulnerable to Simple Yet Dangerous Attacks. arXiv:2502.08586 [cs.LG] <https://arxiv.org/abs/2502.08586>
- [55] Tong Liu, Zizhuang Deng, Guozhu Meng, Yuekang Li, and Kai Chen. 2024. Demystifying RCE Vulnerabilities in LLM-Integrated Apps. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security (CCS '24)*. ACM, 1716–1730. doi:10.1145/3658644.3690338
- [56] Xiaogeng Liu, Nan Xu, Muhao Chen, and Chaowei Xiao. 2024. AutoDAN: Generating Stealthy Jailbreak Prompts on Aligned Large Language Models. In *The Twelfth International Conference on Learning Representations*. <https://openreview.net/forum?id=7Jwpw4qKkb>
- [57] Yupei Liu, Yuqi Jia, Jinyuan Jia, Dawn Song, and Neil Zhenqiang Gong. 2025. DataSentinel: A Game-Theoretic Detection of Prompt Injection Attacks. In *2025 IEEE Symposium on Security and Privacy (SP)*. IEEE Computer Society, Los Alamitos, CA, USA, 2190–2208. doi:10.1109/SP61157.2025.00250
- [58] Yedidel Louck, Ariel Stulman, and Amit Dvir. 2025. Improving Google A2A Protocol: Protecting Sensitive Data and Mitigating Unintended Harms in Multi-Agent Systems. arXiv:2505.12490 [cs.CR] <https://arxiv.org/abs/2505.12490>
- [59] Weidi Luo, Shenghong Dai, Xiaogeng Liu, Suman Banerjee, Huan Sun, Muhao Chen, and Chaowei Xiao. 2025. AGrail: A Lifelong Agent Guardrail with Effective and Adaptive Safety Detection. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar (Eds.). Association for Computational Linguistics, Vienna, Austria, 8104–8139. doi:10.18653/v1/2025.acl-long.399
- [60] Matteo Lupinacci, Francesco Aurelio Pironi, Francesco Blefari, Francesco Romeo, Luigi Arena, and Angelo Furfaro. 2025. The Dark Side of LLMs: Agent-based Attacks for Complete Computer Takeover. arXiv:2507.06850 [cs.CR] <https://arxiv.org/abs/2507.06850>
- [61] Microsoft. 2023. Introducing Microsoft 365 Copilot – your copilot for work. <https://blogs.microsoft.com/blog/2023/03/16/introducing-microsoft-365-copilot-your-copilot-for-work/>. Accessed: 2026-06-12.
- [62] João Moura. 2023. CrewAI: Framework for orchestrating role-playing, autonomous AI agents. <https://github.com/crewAIInc/crewAI>. Accessed: 2026-06-12.
- [63] Milad Nasr, Nicholas Carlini, Chawin Sitawarin, Sander V. Schulhoff, Jamie Hayes, Michael Ilie, Juliette Pluto, Shuang Song, Harsh Chaudhari, Ilya Shumailov, Abhradeep Thakurta, Kai Yuanqing Xiao, Andreas Terzis, and Florian Tramèr. 2025. The Attacker Moves Second: Stronger Adaptive Attacks Bypass Defenses Against Llm Jailbreaks and Prompt Injections. arXiv:2510.09023 [cs.LG] <https://arxiv.org/abs/2510.09023>
- [64] National Institute of Standards and Technology. 2025. CVE-2025-53773 Detail - NVD. <https://nvd.nist.gov/vuln/detail/CVE-2025-53773>. National Vulnerability Database.
- [65] Marc Ohm, Henrik Plate, Arnold Sykosch, and Michael Meier. 2020. Backstabber’s Knife Collection: A Review of Open Source Software Supply Chain Attacks. In *Detection of Intrusions and Malware, and Vulnerability Assessment: 17th International Conference, DIMVA 2020, Lisbon, Portugal, June 24–26, 2020, Proceedings* (Lisbon, Portugal). Springer-Verlag, Berlin, Heidelberg, 23–43. doi:10.1007/978-3-030-52683-2\_2
- [66] OpenAI. 2023. Assistants API Documentation. <https://platform.openai.com/docs/assistants>. Accessed: 2026-06-12.
- [67] Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Christiano, Jan Leike, and Ryan Lowe. 2022. Training language models to follow instructions with human feedback. In *Proceedings of the 36th International Conference on Neural Information Processing Systems* (New Orleans, LA, USA) (NIPS '22). Curran Associates Inc., Red Hook, NY, USA, Article 2011, 15 pages.
- [68] OWASP Foundation. 2025. OWASP Top 10 for Large Language Model Applications. <https://genai.owasp.org/llm-top-10/>. Accessed: 2026-06-12.



- [69] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. 2024. MemGPT: Towards LLMs as Operating Systems. arXiv:2310.08560 [cs.AI] <https://arxiv.org/abs/2310.08560>
- [70] Rodrigo Pedro, Miguel E. Coimbra, Daniel Castro, Paulo Carreira, and Nuno Santos. 2025. Prompt-to-SQL Injections in LLM-Integrated Web Applications: Risks and Defenses. In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. 1768–1780. doi:10.1109/ICSE55347.2025.00007
- [71] Fábio Perez and Ian Ribeiro. 2022. Ignore Previous Prompt: Attack Techniques For Language Models. arXiv:2211.09527 [cs.CL] <https://arxiv.org/abs/2211.09527>
- [72] ProtectAI. 2024. protectai/deberta-v3-base-prompt-injection. <https://huggingface.co/protectai/deberta-v3-base-prompt-injection>. Computer software.
- [73] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. 2019. Language Models are Unsupervised Multitask Learners. *OpenAI* (2019). [https://cdn.openai.com/better-language-models/language\\_models\\_are\\_unsupervised\\_multitask\\_learners.pdf](https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf)
- [74] Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. 2023. Direct preference optimization: your language model is secretly a reward model. In *Proceedings of the 37th International Conference on Neural Information Processing Systems (New Orleans, LA, USA) (NIPS '23)*. Curran Associates Inc., Red Hook, NY, USA, Article 2338, 14 pages.
- [75] Sippo Rossi, Alisia Marianne Michel, Raghava Rao Mukkamala, and Jason Bennett Thatcher. 2024. An Early Categorization of Prompt Injection Attacks on Large Language Processing Models. arXiv:2402.00898 [cs.CR] <https://arxiv.org/abs/2402.00898>
- [76] Yangjun Ruan, Honghua Dong, Andrew Wang, Silviu Pitis, Yongchao Zhou, Jimmy Ba, Yann Dubois, Chris J. Maddison, and Tatsunori Hashimoto. 2024. Identifying the Risks of LM Agents with an LM-Emulated Sandbox. In *The Twelfth International Conference on Learning Representations*. <https://openreview.net/forum?id=GEwtMk1uA>
- [77] Mark Russinovich, Ahmed Salem, and Ronen Eldan. 2025. Great, now write an article about that: the crescendo multi-turn LLM jailbreak attack. In *Proceedings of the 34th USENIX Conference on Security Symposium (Seattle, WA, USA) (SEC '25)*. USENIX Association, USA, Article 125, 20 pages.
- [78] Spyridon Samonas and David Coss. 2014. The CIA Strikes Back: Redefining Confidentiality, Integrity and Availability in Security. *Journal of Information Systems Security* 10, 3 (2014), 21–45. <https://www.jissec.org/Contents/V10/N3/V10N3-Samonas.html>
- [79] Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language Models Can Teach Themselves to Use Tools. In *Thirty-seventh Conference on Neural Information Processing Systems*. <https://openreview.net/forum?id=Yacmpz84TH>
- [80] Sander Schulhoff, Jeremy Pinto, Ansum Khan, Louis-François Bouchard, Chenglei Si, Svetlana Anati, Valen Tagliabue, Anson Kost, Christopher Carnahan, and Jordan Boyd-Graber. 2023. Ignore This Title and HackAPrompt: Exposing Systemic Vulnerabilities of LLMs Through a Global Prompt Hacking Competition. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, Singapore, 4945–4977. doi:10.18653/v1/2023.emnlp-main.302
- [81] Asif Shahriar, Md Nafiu Rahman, Sadif Ahmed, Farig Sadeque, and Md Rizwan Parvez. 2025. A Survey on Agentic Security: Applications, Threats and Defenses. arXiv:2510.06445 [cs.CL] <https://arxiv.org/abs/2510.06445>
- [82] Yijia Shao, Tianshi Li, Weiyan Shi, Yanchen Liu, and Diyi Yang. 2024. PrivacyLens: Evaluating Privacy Norm Awareness of Language Models in Action. In *The Thirty-eight Conference on Neural Information Processing Systems Datasets and Benchmarks Track*. <https://openreview.net/forum?id=CxNXoMnCKc>
- [83] Tianneng Shi, Jingxuan He, Zhun Wang, Hongwei Li, Linyu Wu, Wenbo Guo, and Dawn Song. 2026. Progent: Securing AI Agents with Privilege Control. arXiv:2504.11703 [cs.CR] <https://arxiv.org/abs/2504.11703>
- [84] Tianneng Shi, Kaijie Zhu, Zhun Wang, Yuqi Jia, Will Cai, Weida Liang, Haonan Wang, Hend Alzahrani, Joshua Lu, Kenji Kawaguchi, Basel Alomair, Xuandong Zhao, William Yang Wang, Neil Gong, Wenbo Guo, and Dawn Song. 2025. PromptArmor: Simple yet Effective Prompt Injection Defenses. arXiv:2507.15219 [cs.CR] <https://arxiv.org/abs/2507.15219>
- [85] Rao Surapaneni, Miku Jha, Michael Vakoc, and Todd Segal. 2025. Announcing the Agent2Agent Protocol (A2A). <https://developers.googleblog.com/en/a2a-a-new-era-of-agent-interopability/>. Google for Developers Blog. Accessed: 2026-06-12.
- [86] Yaxin Tang, Yijia Liu, Jiahe Lan, Zheng Yan, and Erol Gelenbe. 2026. Security of LLM-based agents regarding attacks, defenses, and applications: A comprehensive survey. *Information Fusion* 127 (2026), 103941. doi:10.1016/j.inffus.2025.103941
- [87] Florian Tramer and Dan Boneh. 2019. Slalom: Fast, Verifiable and Private Execution of Neural Networks in Trusted Hardware. In *International Conference on Learning Representations*. <https://openreview.net/forum?id=rJVorjCcKQ>
- [88] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention is All you Need. In *Advances in Neural Information Processing Systems*. I. Guyon, U. Von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett (Eds.), Vol. 30. Curran Associates, Inc.

- [https://proceedings.neurips.cc/paper\\_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf](https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf)
- [89] Eric Wallace, Kai Xiao, Reimar Leike, Lilian Weng, Johannes Heidecke, and Alex Beutel. 2024. The Instruction Hierarchy: Training LLMs to Prioritize Privileged Instructions. arXiv:2404.13208 [cs.CR] <https://arxiv.org/abs/2404.13208>
  - [90] Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, Wayne Xin Zhao, Zhewei Wei, and Jirong Wen. 2024. A survey on large language model based autonomous agents. *Frontiers of Computer Science* 18, 6 (March 2024). doi:10.1007/s11704-024-40231-1
  - [91] Shilong Wang, Guibin Zhang, Miao Yu, Guancheng Wan, Fanci Meng, Chongye Guo, Kun Wang, and Yang Wang. 2025. G-Safeguard: A Topology-Guided Security Lens and Treatment on LLM-based Multi-agent Systems. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, Vienna, Austria, 7261–7276. doi:10.18653/v1/2025.acl-long.359
  - [92] Yifei Wang, Dizhan Xue, Shengjie Zhang, and Shengsheng Qian. 2024. BadAgent: Inserting and Activating Backdoor Attacks in LLM Agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, Bangkok, Thailand, 9811–9827. doi:10.18653/v1/2024.acl-long.530
  - [93] Alexander Wei, Nika Haghtalab, and Jacob Steinhardt. 2023. Jailbroken: How Does LLM Safety Training Fail?. In *Thirty-seventh Conference on Neural Information Processing Systems*. <https://openreview.net/forum?id=JA235JGM09>
  - [94] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. 2022. Chain-of-thought prompting elicits reasoning in large language models. In *Proceedings of the 36th International Conference on Neural Information Processing Systems (New Orleans, LA, USA) (NIPS '22)*. Curran Associates Inc., Red Hook, NY, USA, Article 1800, 14 pages.
  - [95] Roy Weiss, Daniel Ayzenshteyn, Guy Amit, and Yisroel Mirsky. 2024. What Was Your Prompt? A Remote Keylogging Attack on AI Assistants. In *33rd USENIX Security Symposium (USENIX Security 24)*. USENIX Association, Philadelphia, PA, 3367–3384. <https://www.usenix.org/conference/usenixsecurity24/presentation/weiss>
  - [96] Simon Willison. 2023. Delimiters won't save you from prompt injection. <https://simonwillison.net/2023/May/11/delimiters-wont-save-you/>. Accessed: 2026-06-12.
  - [97] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W White, Doug Burger, and Chi Wang. 2024. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversations. In *First Conference on Language Modeling*. <https://openreview.net/forum?id=BAakY1hNKS>
  - [98] Yuhao Wu, Franziska Roesner, Tadayoshi Kohno, Ning Zhang, and Umar Iqbal. 2025. IsolateGPT: An Execution Isolation Architecture for LLM-Based Agentic Systems. arXiv:2403.04960 [cs.CR] <https://arxiv.org/abs/2403.04960>
  - [99] Zhiheng Xi, Wenxiang Chen, Xin Guo, Wei He, Yiwen Ding, Boyang Hong, Ming Zhang, Junzhe Wang, Senjie Jin, Enyu Zhou, Rui Zheng, Xiaoran Fan, Xiao Wang, Limao Xiong, Yuhao Zhou, Weiran Wang, Changhao Jiang, Yicheng Zou, Xiangyang Liu, Zhangyue Yin, Shihan Dou, Rongxiang Weng, Wensen Cheng, Qi Zhang, Wenjuan Qin, Yongyan Zheng, Xipeng Qiu, Xuanjing Huang, and Tao Gui. 2023. The Rise and Potential of Large Language Model Based Agents: A Survey. arXiv:2309.07864 [cs.AI] <https://arxiv.org/abs/2309.07864>
  - [100] Zhen Xiang, Linzhi Zheng, Yanjie Li, Junyuan Hong, Qinbin Li, Han Xie, Jiawei Zhang, Zidi Xiong, Chulin Xie, Carl Yang, Dawn Song, and Bo Li. 2025. GuardAgent: Safeguard LLM Agents via Knowledge-Enabled Reasoning. In *Forty-second International Conference on Machine Learning*. <https://openreview.net/forum?id=2nBcjCZrrP>
  - [101] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. 2023. Tree of thoughts: deliberate problem solving with large language models. In *Proceedings of the 37th International Conference on Neural Information Processing Systems (New Orleans, LA, USA) (NIPS '23)*. Curran Associates Inc., Red Hook, NY, USA, Article 517, 14 pages.
  - [102] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. arXiv:2210.03629 [cs.CL] <https://arxiv.org/abs/2210.03629>
  - [103] Miao Yu, Fanci Meng, Xinyun Zhou, Shilong Wang, Junyuan Mao, Linsey Pan, Tianlong Chen, Kun Wang, Xinfeng Li, Yongfeng Zhang, Bo An, and Qingsong Wen. 2025. A Survey on Trustworthy LLM Agents: Threats and Countermeasures. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2 (Toronto ON, Canada) (KDD '25)*. Association for Computing Machinery, New York, NY, USA, 6216–6226. doi:10.1145/3711896.3736561
  - [104] Tongxin Yuan, Zhiwei He, Lingzhong Dong, Yiming Wang, Ruijie Zhao, Tian Xia, Lizhen Xu, Binglin Zhou, Li Fangqi, Zhuosheng Zhang, Rui Wang, and Gongshen Liu. 2024. R-Judge: Benchmarking Safety Risk Awareness for LLM Agents. In *ICLR 2024 Workshop on Large Language Model (LLM) Agents*. <https://openreview.net/forum?id=g6Yy4YXrU>
  - [105] Qiusi Zhan, Zhixiang Liang, Zifan Ying, and Daniel Kang. 2024. InjecAgent: Benchmarking Indirect Prompt Injections in Tool-Integrated Large Language Model Agents. In *Findings of the Association for Computational Linguistics: ACL 2024*. Association for Computational Linguistics, Bangkok, Thailand, 10471–10506. doi:10.18653/v1/2024.findings-acl.624

- [106] Hanrong Zhang, Jingyuan Huang, Kai Mei, Yifei Yao, Zhenting Wang, Chenlu Zhan, Hongwei Wang, and Yongfeng Zhang. 2025. Agent Security Bench (ASB): Formalizing and Benchmarking Attacks and Defenses in LLM-based Agents. In *The Thirteenth International Conference on Learning Representations*. <https://openreview.net/forum?id=V4y0CpX4hK>
- [107] Yuanhe Zhang, Weiliu Wang, Zhenhong Zhou, Kun Wang, Jie Zhang, Li Sun, Yang Liu, and Sen Su. 2025. LeechHijack: Covert Computational Resource Exploitation in Intelligent Agent Systems. arXiv:2512.02321 [cs.CR] <https://arxiv.org/abs/2512.02321>
- [108] Peter Yong Zhong, Siyuan Chen, Ruiqi Wang, McKenna McCall, Ben L. Titzer, Heather Miller, and Phillip B. Gibbons. 2025. RTBAS: Defending LLM Agents Against Prompt Injection and Privacy Leakage. arXiv:2502.08966 [cs.CR] <https://arxiv.org/abs/2502.08966>
- [109] Zexuan Zhong, Ziqing Huang, Alexander Wettig, and Danqi Chen. 2023. Poisoning Retrieval Corpora by Injecting Adversarial Passages. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Association for Computational Linguistics, 13764–13775. doi:10.18653/v1/2023.emnlp-main.849
- [110] Zhenhong Zhou, Zherui Li, Jie Zhang, Yuanhe Zhang, Kun Wang, Yang Liu, and Qing Guo. 2025. CORBA: Contagious Recursive Blocking Attacks on Multi-Agent Systems Based on Large Language Models. arXiv:2502.14529 [cs.CL] <https://arxiv.org/abs/2502.14529>
- [111] Kaijie Zhu, Xianjun Yang, Jindong Wang, Wenbo Guo, and William Yang Wang. 2025. MELON: Provable Defense Against Indirect Prompt Injection Attacks in AI Agents. In *Forty-second International Conference on Machine Learning*. <https://openreview.net/forum?id=gt1MmGaKdZ>
- [112] Pengyu Zhu, Zhenhong Zhou, Yuanhe Zhang, Shilinlu Yan, Kun Wang, and Sen Su. 2025. DemonAgent: Dynamically Encrypted Multi-Backdoor Implantation Attack on LLM-based Agent. In *Findings of the Association for Computational Linguistics: EMNLP 2025*. Association for Computational Linguistics, Suzhou, China, 2890–2912. doi:10.18653/v1/2025.findings-emnlp.157
- [113] Andy Zou, Zifan Wang, Nicholas Carlini, Milad Nasr, J. Zico Kolter, and Matt Fredrikson. 2023. Universal and Transferable Adversarial Attacks on Aligned Language Models. arXiv:2307.15043 [cs.CL] <https://arxiv.org/abs/2307.15043>
- [114] Wei Zou, Runpeng Geng, Binghui Wang, and Jinyuan Jia. 2025. PoisonedRAG: Knowledge Corruption Attacks to Retrieval-Augmented Generation of Large Language Models. In *34th USENIX Security Symposium (USENIX Security 25)*. USENIX Association, Seattle, WA, 3827–3844. <https://www.usenix.org/conference/usenixsecurity25/presentation/zou-poisonedrag>
- [115] Egor Zverev, Sahar Abdelnabi, Soroush Tabesh, Mario Fritz, and Christoph H. Lampert. 2025. Can LLMs Separate Instructions From Data? And What Do We Even Mean By That?. In *The Thirteenth International Conference on Learning Representations*. <https://openreview.net/forum?id=8EtSBX41mt>